Tutorial frontEndflask con jinja2

Paso 1: Preparar Git y GitHub para el proyecto
Antes de escribir una sola línea de código, conviene preparar el repositorio.

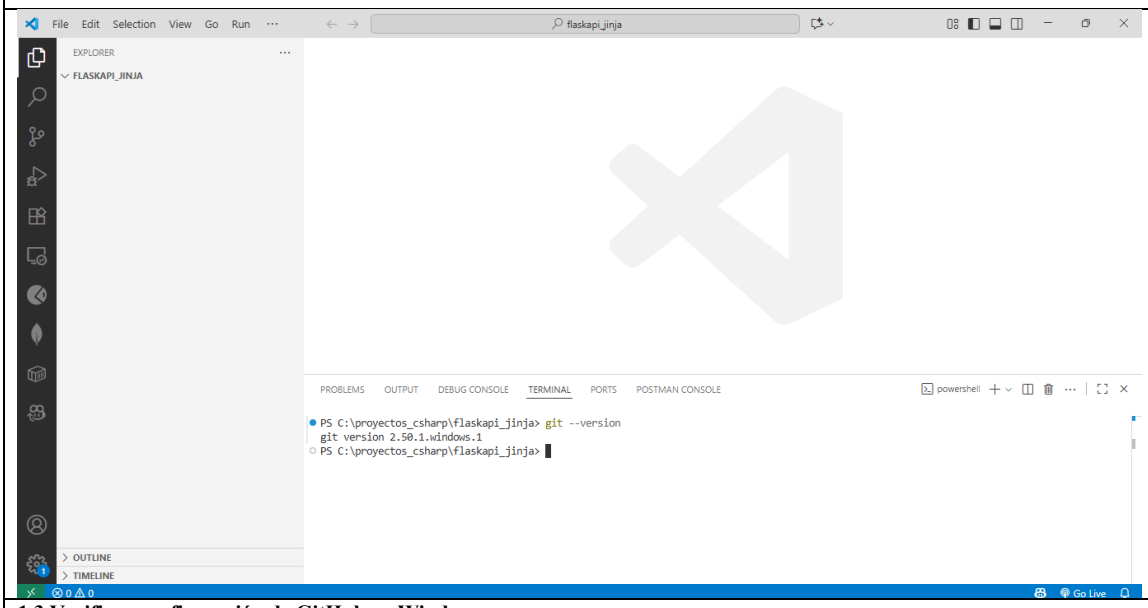
1.1 Crear la carpeta del proyecto en la máquina
En la terminal, elegir la ubicación donde quieras guardar tu proyecto y ejecutar:

```
mkdir frontflask_jinja

cd frontflask_jinja
```

1.2 Instalar Git y verificar instalación

1. **Instalar Git en Windows**
   - Ir a la página oficial: https://git-scm.com/downloads
   - Descargar el instalador para Windows.
   - Durante la instalación, dejar las opciones por defecto (a menos que quieras personalizar).
2. **Verificar instalación**
   - Abrir la terminal (PowerShell o CMD).
   - Ejecutar:
   - git --version
   - Si todo está correcto, deberías ver algo como:
   - git version 2.47.0.windows.1

Eso significa que Git está instalado y listo.



1.3 Verificar configuración de GitHub en Windows

1.3.1 Abrir el Administrador de credenciales

1. Presiona la tecla Windows y escribe:
2. Administrador de credenciales
3. Haz clic en la aplicación que aparece con ese nombre.

1.3.2 Ir a "Credenciales de Windows"

- Dentro del Administrador de credenciales, selecciona la pestaña **Credenciales de Windows**.

1.3.3 Buscar credenciales de GitHub

- En la lista, busca entradas que digan:
  - github.com
  - o git:https://github.com

1.3.4 Verificar usuario

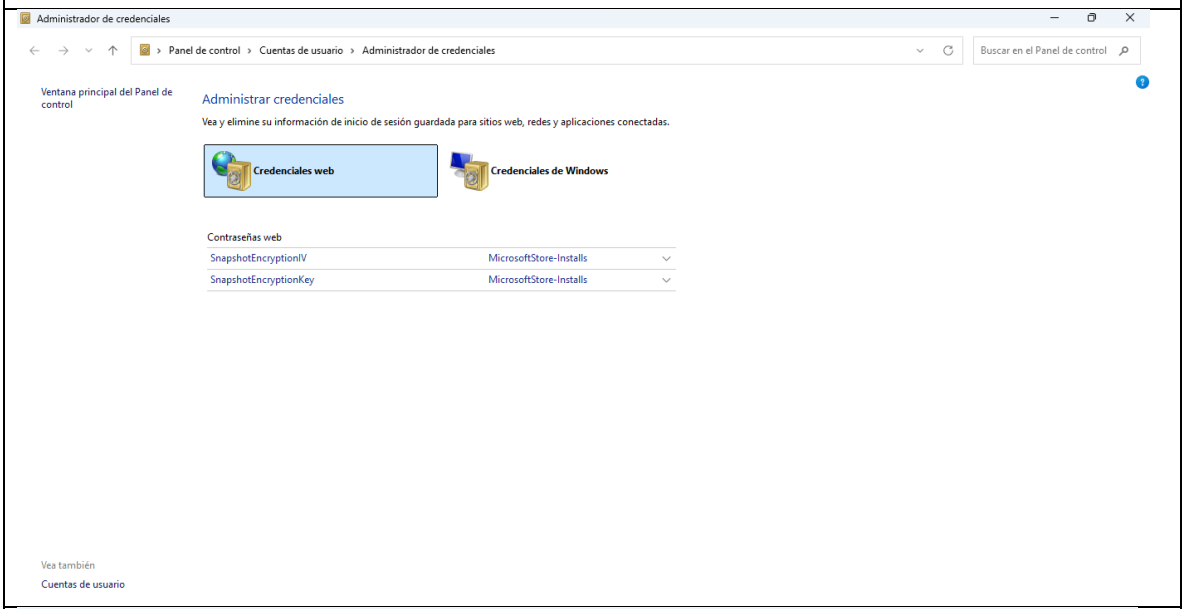
- Haz clic en la entrada de github.com.
- Ahí se mostrará el usuario (si es tu cuenta, está bien).

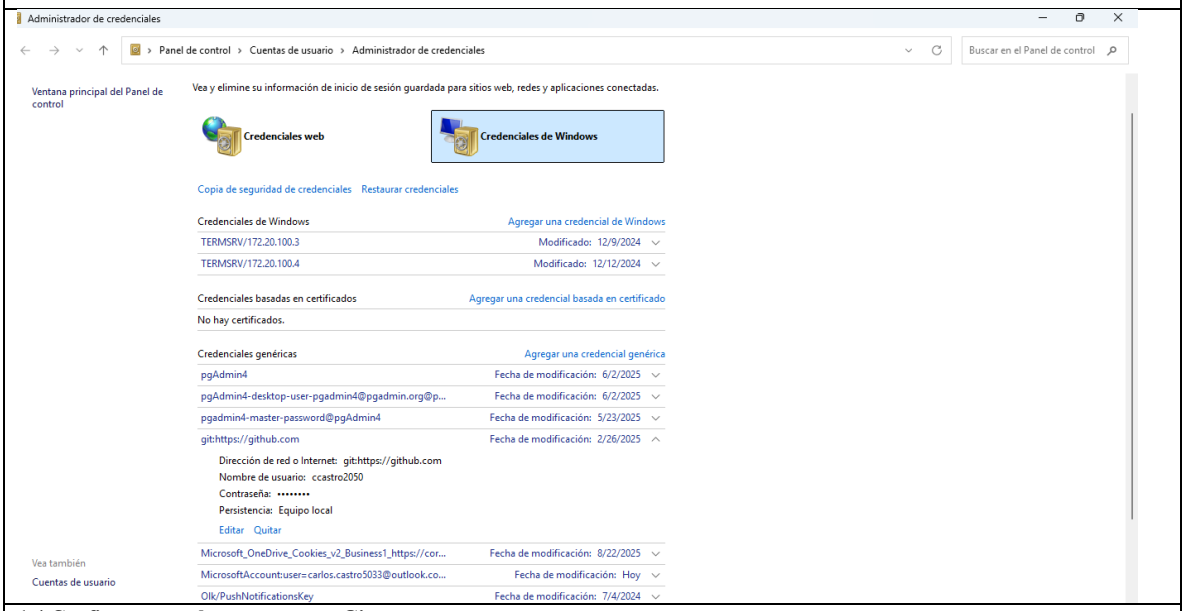
- Si ves que pertenece a otra persona, deberás **quitarla**.

1.3.5 Quitar credenciales incorrectas

- Selecciona la entrada.
- Pulsa el botón **Quitar**.
- Repite si hay varias entradas que no son tuyas.

Resultado esperado
Ahora tu Windows está libre de credenciales antiguas de GitHub.
La próxima vez que uses git push, Git te pedirá usuario y contraseña/token. Ahí pondrás tu usuario real de GitHub y el token personal (PAT).





1.4 Configurar nombre y correo en Git

Explicación
Git necesita saber tu nombre y tu correo para firmar cada commit que hagas.
El correo debe coincidir con el que usas en GitHub (puede ser tu correo real o el correo privado que GitHub te da).

1.4.1 Configurar variables globales

En la terminal escribe (cambia por tus datos reales):
git config --global user.name "Tu Nombre"

git config --global user.email "tu_correo@ejemplo.com"
Ejemplo:
git config --global user.name "Carlos Castro"

git config --global user.email "carlosarturo.castrocastrocastro@gmail.com"

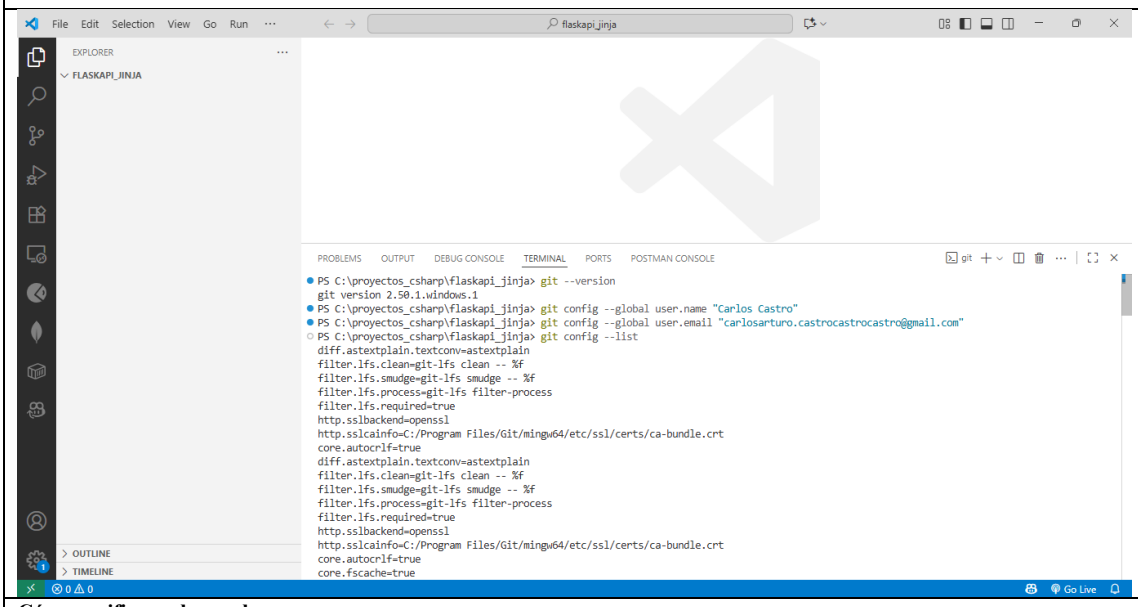
1.4.2 Verificar configuración
Revisa que quedó guardado:
git config --list
Debe aparecer:
user.name=Carlos Castro
user.email= carlosarturo.castrocastrocastro@gmail.com

1.4.3 Nota importante

- Si pones --global, la configuración aplica a **todos los proyectos en tu PC**.
- Si prefieres que solo aplique a este proyecto, quita el --global:

git config user.name "Carlos Castro"

git config user.email " carlosarturo.castrocastrocastro@gmail.com "

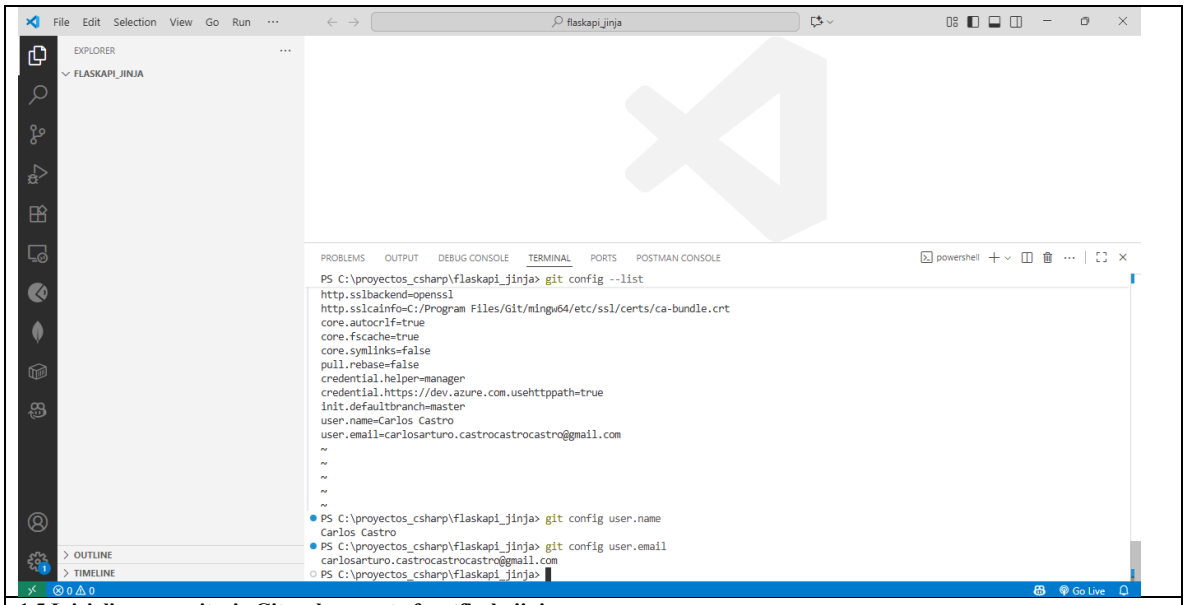


Cómo verificar solo nombre y correo
Ejecuta estos tres comandos uno por uno:

q es para salir

git config user.name

git config user.email
La salida debería ser:
Carlos Castro
carlosarturo.castrocastrocastro@gmail.com



1.5 Inicializar repositorio Git en la carpeta frontflask_jinja

Explicación

Hasta ahora solo hemos preparado Git en tu máquina (instalación, credenciales y configuración).
Ahora toca decirle a Git: *"esta carpeta frontflask_jinja es un repositorio, empieza a rastrear mis archivos"*.
Eso se hace creando una carpeta oculta .git dentro del proyecto.

1.5.1 Inicializar el repositorio

En la terminal, estando dentro de la carpeta frontflask_jinja, escribe:

```
git init
```

Si todo va bien, verás un mensaje como:
Initialized empty Git repository in C:/proyectos_csharp/frontflask_jinja/.git/

1.5.2 Verificar estado inicial

Después ejecuta:

```
git status
```

Debe mostrar algo parecido a:
On branch master
No commits yet
nothing to commit
Eso significa que Git ya está funcionando pero todavía no hay commits.

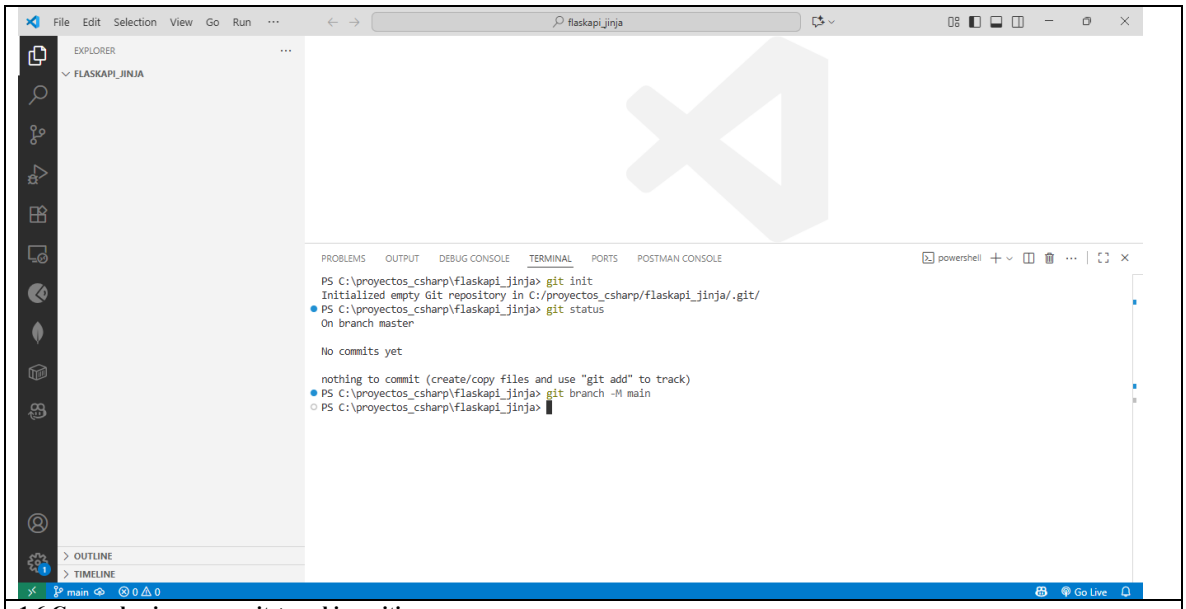
1.5.3 Renombrar la rama a "main"

Git por defecto crea la rama **master**, pero GitHub ahora usa **main** como estándar.
Para evitar problemas luego, renombra la rama con:

```
git branch -M main
```

Resultado esperado de 1.5

- Tu carpeta frontflask_jinja ya es un repositorio Git local.
- La rama principal se llama **main** (lista para conectarla con GitHub después).



1.6 Crear el primer commit + archivo .gitignore
El archivo .gitignore le dice a Git qué carpetas o archivos **no debe rastrear ni subir a GitHub**.
En proyectos Python con Flask normalmente se ignoran:

- Entornos virtuales (.venv/ o venv/)
- Archivos compilados (__pycache__/, *.pyc)
- Archivos temporales (.DS_Store, Thumbs.db)
- Configuración personal de VS Code (.vscode/)

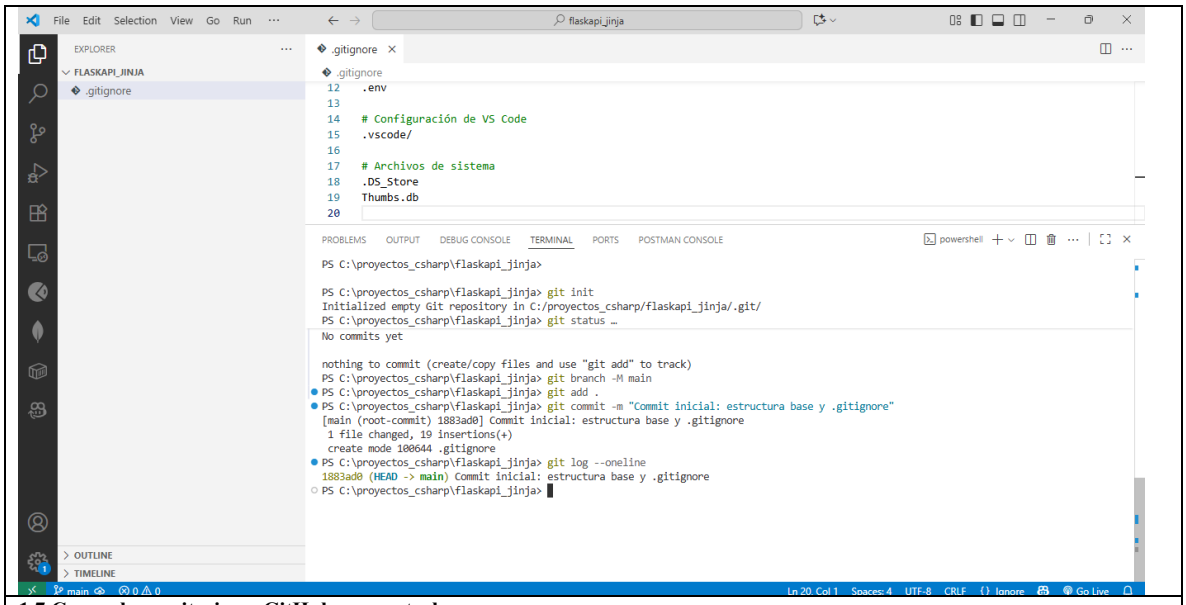
1.6.1 Crear el archivo .gitignore
En la raíz de tu proyecto (frontflask_jinja) crea un archivo llamado:
.gitignore
Y adentro escribe lo siguiente:

```
# Entornos virtuales
venv/
.venv/

# Archivos compilados de Python
__pycache__/
*.pyc
*.pyo
*.pyd

# Archivos de entorno
.env

# Configuración de VS Code
.vscode/

# Archivos de sistema
.DS_Store
Thumbs.db
```

1.6.2 Agregar archivos al staging
En la terminal, estando en la carpeta del proyecto:
git add .

1.6.3 Crear el commit inicial
git commit -m "Commit inicial: estructura base y .gitignore"

1.6.4 Verificar
git log --oneline
Debe aparecer al menos 1 commit con el mensaje que escribiste.



1.7 Crear el repositorio en GitHub y conectarlo
Explicación
Hasta ahora el repositorio es **solo local** en tu computadora.
El siguiente paso es **crear un repositorio en GitHub** y enlazarlo con este.

1.7.1 Crear repositorio en GitHub

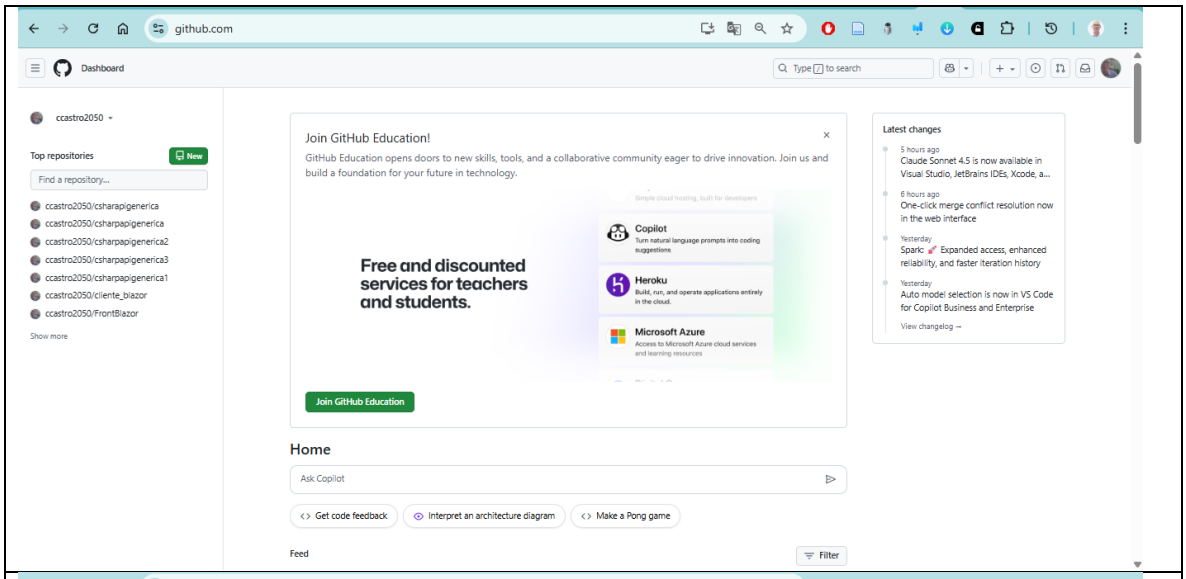
1. Entra en tu cuenta de GitHub.
2. Haz clic en el botón verde **New** (Nuevo repositorio).
3. Pon el nombre del repositorio exactamente igual:
4. **frontflask_jinja**
5. Deja las opciones:
   - Público (o privado si prefieres).
   - **No** inicializar con README (porque ya tienes archivos).
6. Haz clic en **Create repository**.

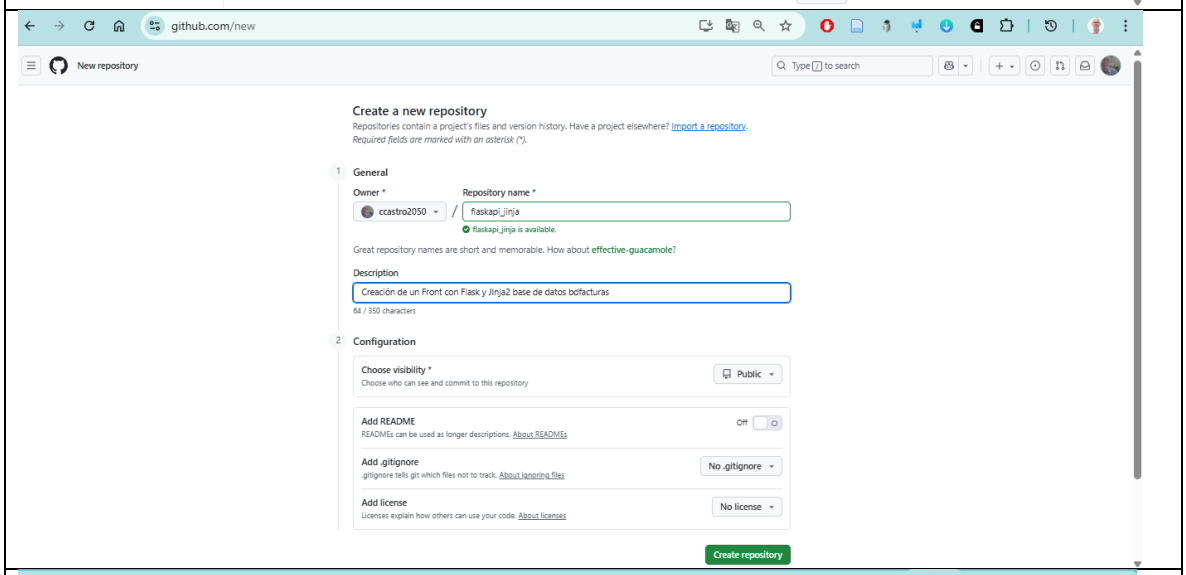
1.7.2 Conectar Git local con GitHub
GitHub te mostrará unos comandos después de crear el repo.
En tu terminal de VS Code (dentro del proyecto), ejecuta:
git remote add origin https://github.com/**TU_USUARIO**/frontflask_jinja.git

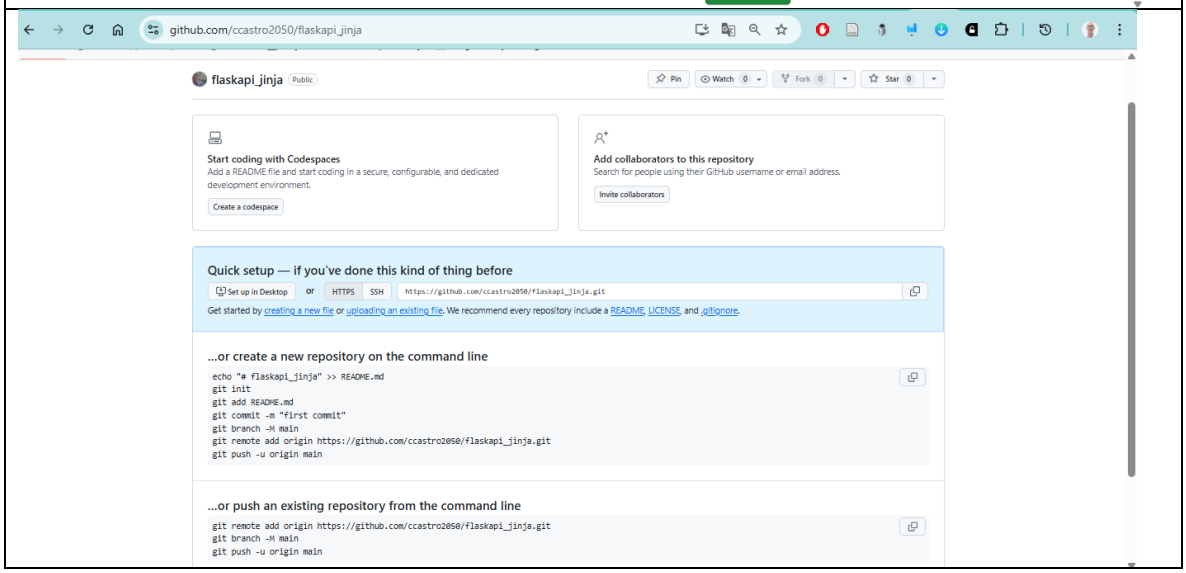
Ejemplo:
git remote add origin https://github.com/ **ccastro2050** /frontflask_jinja.git
(Reemplaza TU_USUARIO con tu usuario real de GitHub)
Si se equivoca en este paso y no configuró el usuario correctamente, ***volver a configuralo***

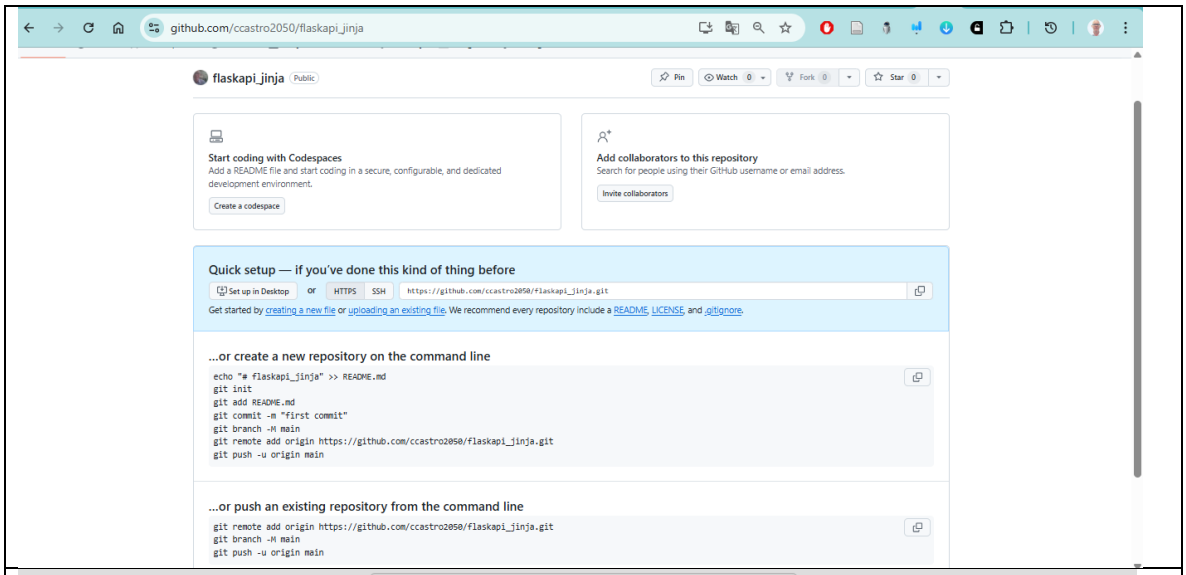
git remote set-url origin https://github.com/**ccastro2050**/frontflask_jinja.git

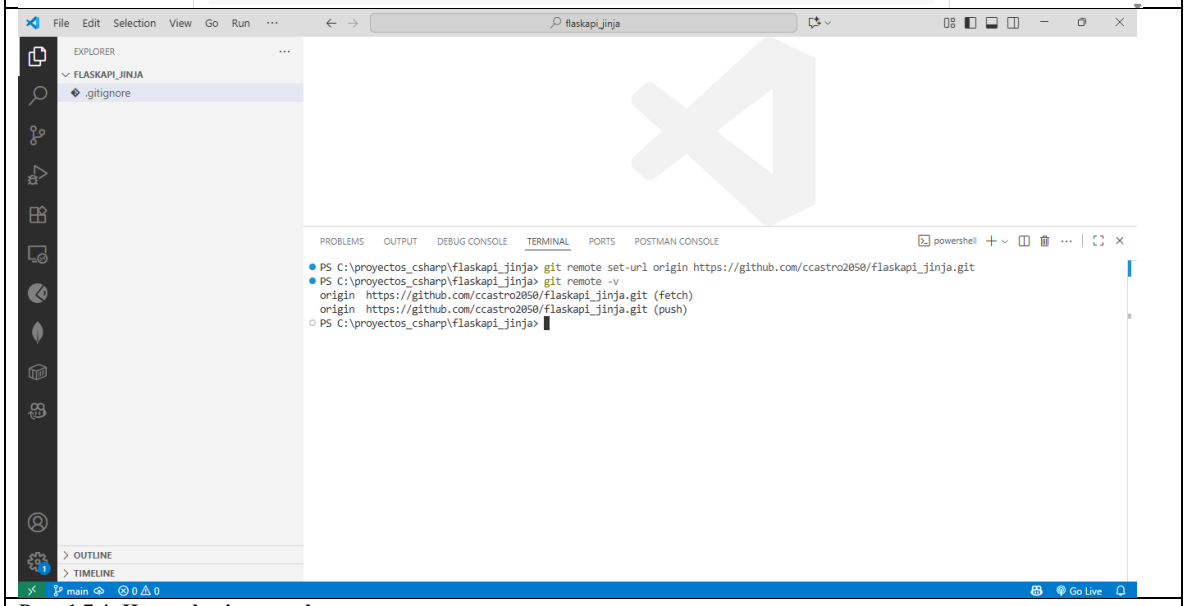
1.7.3 Verificar conexión
git remote -v
Debe mostrar algo como:
origin https://github.com/**TU_USUARIO**/frontflask_jinja.git (fetch)
origin https://github.com/**TU_USUARIO**/frontflask_jinja.git (push)


github.com
Dashboard
Type / to search
ccastro2050
Top repositories
New
Find a repository...
ccastro2050/csharapigenerica
ccastro2050/csharpapigenerica
ccastro2050/csharpapigenerica2
ccastro2050/csharpapigenerica3
ccastro2050/csharpapigenerica1
ccastro2050/cliente_blazor
ccastro2050/FrontBlazor
Show more
Join GitHub Education!
GitHub Education opens doors to new skills, tools, and a collaborative community eager to drive innovation. Join us and build a foundation for your future in technology.
Free and discounted services for teachers and students.
Copilot
Turn natural language prompts into coding suggestions
Heroku
Build, run, and operate applications entirely in the cloud.
Microsoft Azure
Access to Microsoft Azure cloud services and learning resources
Join GitHub Education
Latest changes
5 hours ago
Claude Sonnet 4.5 is now available in Visual Studio, JetBrains IDEs, Xcode, a...
6 hours ago
One-click merge conflict resolution now in the web interface
Yesterday
Spark: 🚀 Expanded access, enhanced reliability, and faster iteration history
Yesterday
Auto model selection is now in VS Code for Copilot Business and Enterprise
View changelog →
Home
Ask Copilot
Get code feedback
Interpret an architecture diagram
Make a Pong game
Feed
Filter


github.com/new
New repository
Type / to search
Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
Required fields are marked with an asterisk (*).
1 General
Owner *
Repository name *
ccastro2050
flaskapi_jinja
flaskapi_jinja is available.
Great repository names are short and memorable. How about effective-guacamole?
Description
Creación de un Front con Flask y Jinja2 base de datos bdfacturas
64 / 350 characters
2 Configuration
Choose visibility *
Choose who can see and commit to this repository
Public
Add README
READMEs can be used as longer descriptions. About READMEs
Off
Add .gitignore
.gitignore tells git which files not to track. About ignoring files
No .gitignore
Add license
Licenses explain how others can use your code. About licenses
No license
Create repository


github.com/ccastro2050/flaskapi_jinja
flaskapi_jinja
Public
Pin
Watch 0
Fork 0
Star 0
Start coding with Codespaces
Add a README file and start coding in a secure, configurable, and dedicated development environment.
Create a codespace
Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators
Quick setup — if you've done this kind of thing before
Set up in Desktop
or
HTTPS
SSH
https://github.com/ccastro2050/flaskapi_jinja.git
Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.
...or create a new repository on the command line
echo "# flaskapi_jinja" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ccastro2050/flaskapi_jinja.git
git push -u origin main
...or push an existing repository from the command line
git remote add origin https://github.com/ccastro2050/flaskapi_jinja.git
git branch -M main
git push -u origin main





Paso 1.7.4: Hacer el primer push

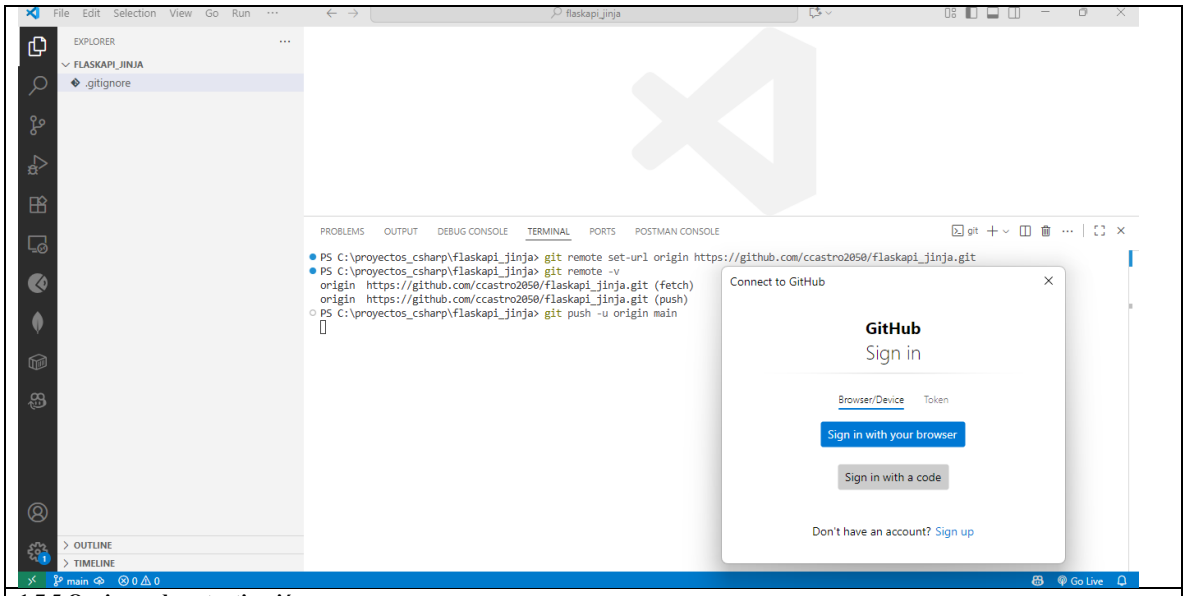
Ahora vamos a subir el commit inicial que ya hiciste a GitHub:

git push -u origin main

- git push → sube los cambios locales al remoto.
- -u → vincula la rama main local con la rama main remota (esto permite que después solo uses git push o git pull sin parámetros).
- origin → nombre del remoto.
- main → rama principal.

Resultado esperado:

Git te pedirá autenticarte en GitHub (si quitaste las credenciales viejas, como ya hicimos en 1.3, debe pedir login/token). Después, en tu repositorio en GitHub (https://github.com/ccastro2050/frontflask_jinja), verás el archivo .gitignore y tu primer commit.



1.7.5 Opciones de autenticación

Tienes 2 formas principales:

Opción A. Navegador (más sencilla y recomendada)

1. Haz clic en **"Sign in with your browser"**.
2. Se abrirá GitHub en tu navegador.
3. Inicia sesión con tu usuario ccastro2050 y tu contraseña normal.
4. Si tienes activado 2FA (autenticación de dos pasos), pedirá tu código.
5. Cuando confirmes, GitHub vinculará tu sesión con Git en Windows.

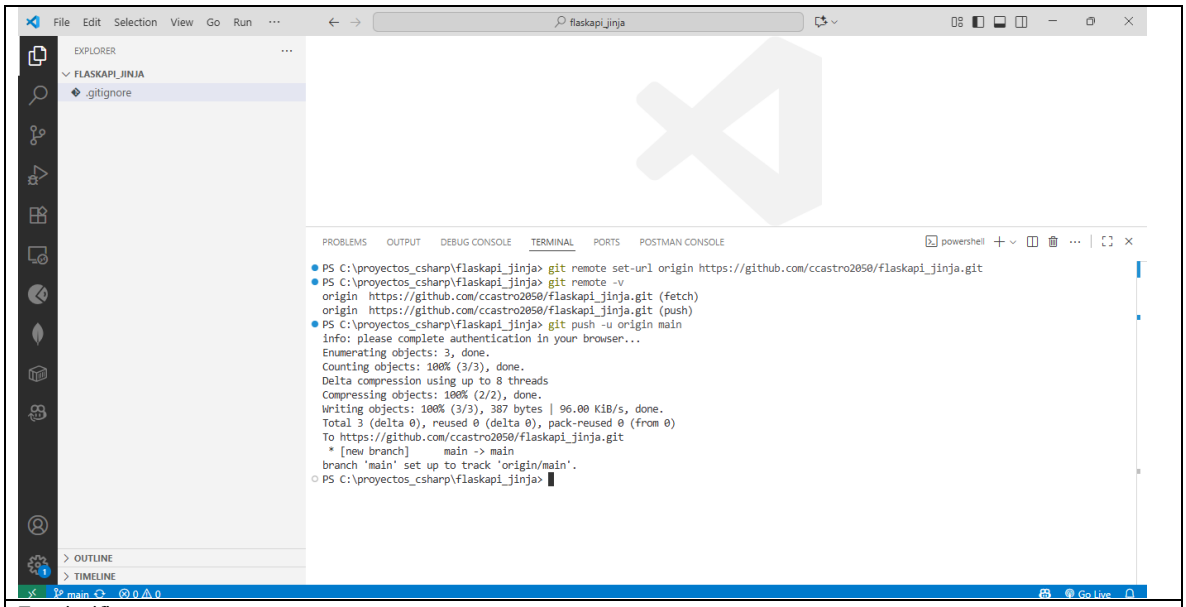
Esto almacena un token temporal en el **Administrador de credenciales de Windows** y ya no tendrás que poner usuario/contraseña cada vez.

Opción B. Token personal (PAT)

1. En GitHub, ve a:
   Settings > Developer settings > Personal access tokens > Tokens (classic)
   o si usas **Fine-grained tokens**, crea uno nuevo.
2. Genera un token con permisos:
   - repo (para trabajar con repositorios).
3. Copia el token (se muestra solo una vez).
4. En la ventana de GitHub selecciona la pestaña **Token** y pega el token.

Recomendación: Usa la opción **Sign in with your browser** porque es más rápida y menos propensa a errores.

Cuando termines el login, la terminal debería mostrar que el push se realizó con éxito.
En GitHub (https://github.com/ccastro2050/frontflask_jinja) deberías ver tu commit inicial con el .gitignore.

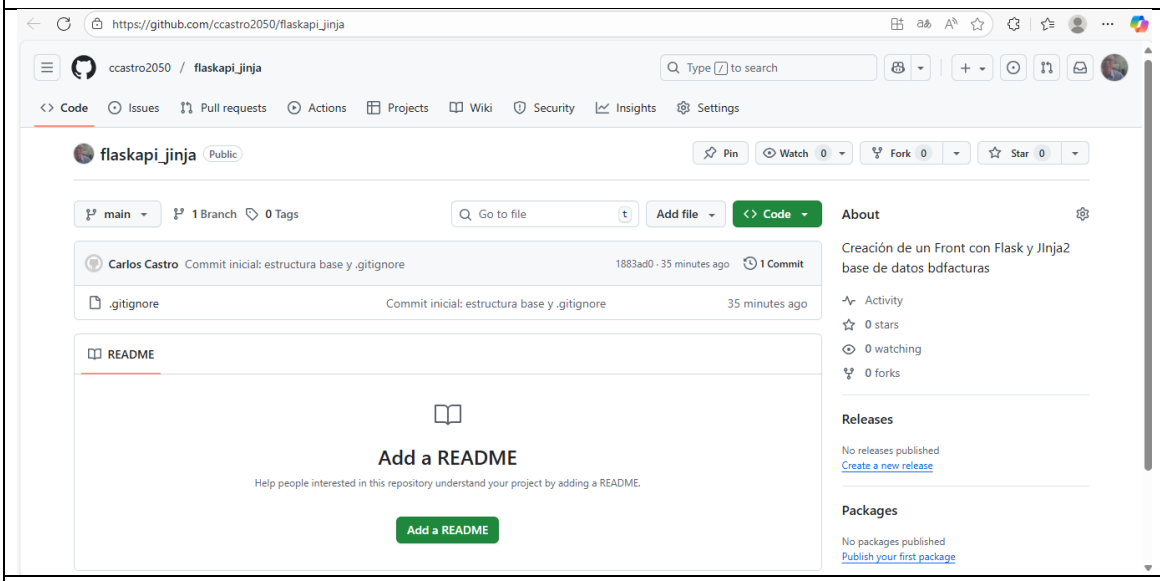


Eso significa que:

- El repositorio local ya está conectado correctamente con el remoto en GitHub.
- La rama main en tu máquina ahora está vinculada con la rama main en GitHub.
- El primer commit (.gitignore) ya fue subido.

Paso 1.7.6 – Verificar en GitHub

1. Abre en tu navegador:
   https://github.com/ccastro2050/frontflask_jinja
2. En la página del repositorio deberías ver el archivo **.gitignore** y el mensaje de commit:
   Commit inicial: estructura base y .gitignore



2. Configuración del proyecto Flask

2.1 Crear la estructura base del proyecto Flask

2.1.1 Crear archivo principal

En la carpeta frontflask_jinja, crear un archivo llamado **app.py** con este contenido básico:

```
from flask import Flask, render_template

aplicacion = Flask(__name__)

@aplicacion.route('/')
def inicio():
```

```
    return render_template('index.html')

if __name__ == '__main__':
    aplicacion.run(debug=True)
```

2.1.2 Crear carpeta de plantillas
En la misma carpeta, crear una subcarpeta llamada **templates**.
Dentro de ella, crear el archivo **index.html** con contenido mínimo:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Proyecto Flask Jinja</title>
</head>
<body>
    <h1>Página inicial del proyecto Flask con Jinja2</h1>
</body>
</html>
```

Paso 2.1.3 — Crear y activar el entorno virtual
Objetivo: aislar las dependencias del proyecto antes de instalar Flask.

1. En la terminal (PowerShell o la de VS Code), ubicarse dentro del proyecto:
2. cd C:\proyectos_csharp\frontflask_jinja
3. Crear el entorno virtual con el siguiente comando:
   python -m venv .venv

Resultado esperado:
Se crea una carpeta llamada .venv dentro del proyecto.

4. Activar el entorno virtual:
   .\.venv\Scripts\Activate

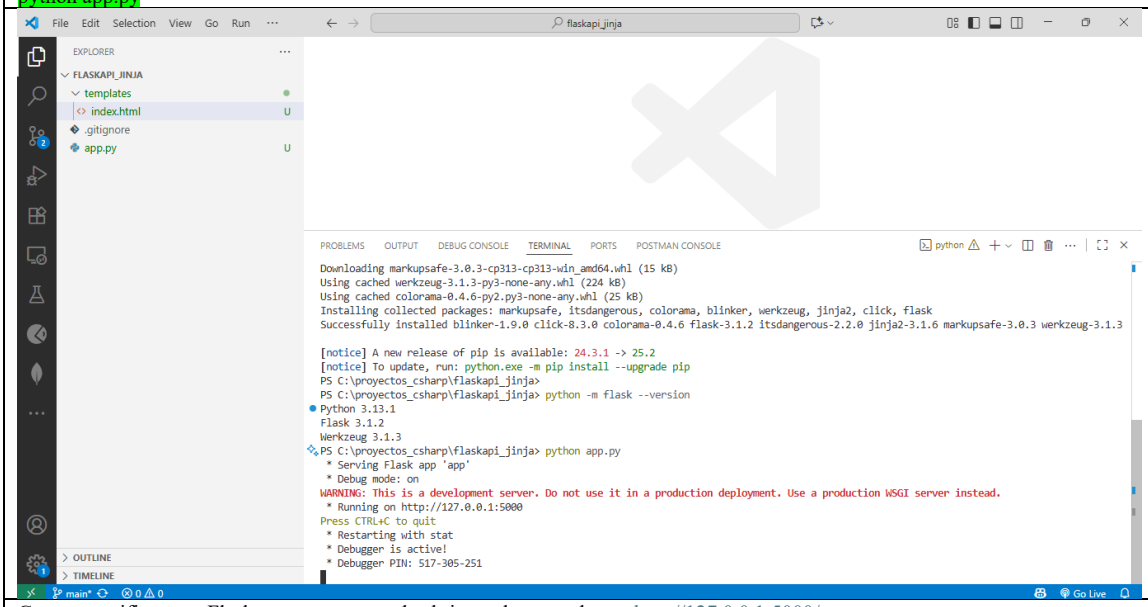
Resultado esperado:
En la terminal aparece algo como:
(.venv) PS C:\proyectos_csharp\frontflask_jinja>

2.1.4 Instalar flsk y verificar instalación de Flask
Ejecuta estos comandos:

```
pip install flask
python -m flask –version
```

2.1.5 Probar servidor Flask básico
Ejecuta este comando para iniciar el servidor con el archivo app.py:

```
python app.py
```



Con eso verificas que Flask arranca y se puede abrir en el navegador en http://127.0.0.1:5000/

Presionar Ctrl + C para parar la ejecución

2.1.6 Reescribir app.py para renderizar index.html con Flask y Jinja2

Reemplazar el contenido de app.py con este código:

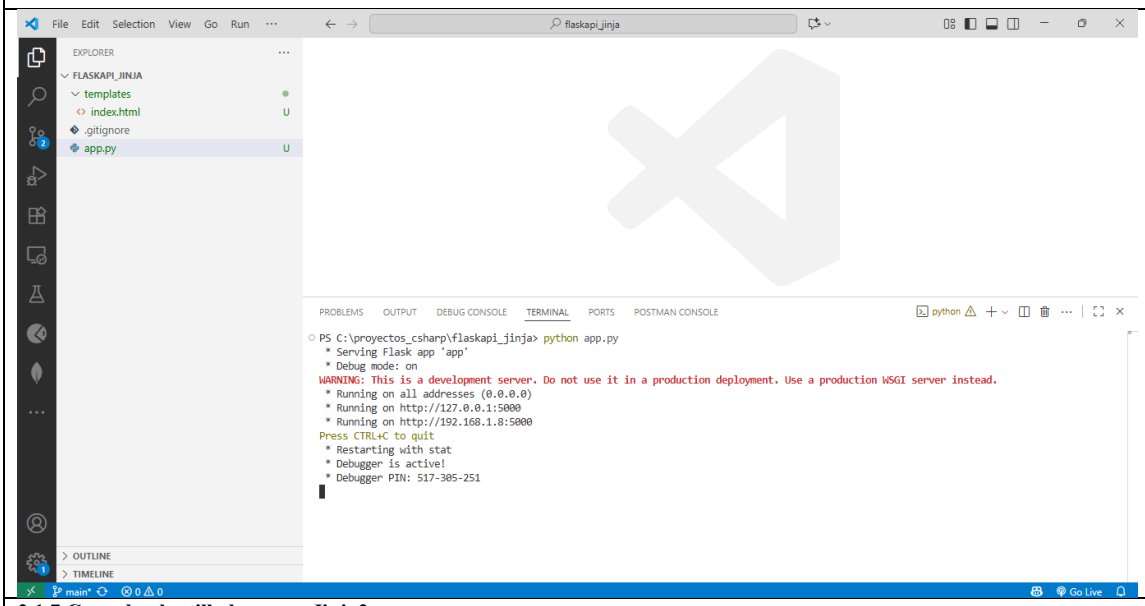
```
# Importar la clase principal de Flask y la función para renderizar plantillas
from flask import Flask, render_template

# Crear la instancia de la aplicación Flask
# __name__ indica a Flask dónde se encuentra el archivo principal
aplicacion = Flask(__name__)

# Definir la ruta principal del sitio (ruta raíz "/")
@aplicacion.route("/")
def inicio():
    """
    Función asociada a la ruta principal.
    Retorna el archivo index.html que debe estar en la carpeta 'templates'.
    """
    return render_template("index.html")

# Punto de entrada de la aplicación
if __name__ == "__main__":
    # Ejecutar la aplicación en modo depuración, en el puerto 5000
    # host="0.0.0.0" hace que se pueda acceder desde la red local
    # debug=True permite reiniciar automáticamente si hay cambios en el código
    aplicacion.run(host="0.0.0.0", port=5000, debug=True)
# Ejecutar la aplicación en modo depuración, en el puerto 5000
```

Explicación

- Flask(__name__) crea la instancia principal de la aplicación.
- @aplicacion.route("/") define la ruta raíz (/).
- render_template("index.html") carga y muestra el archivo index.html que está dentro de la carpeta templates.
- aplicacion.run(debug=True) activa el modo depuración, útil para desarrollo.

El servidor Flask se abre en **http://127.0.0.1:5000** (local).
Si alguien está en tu misma red y accede a http://TU_IP_LOCAL:5000, también verá la aplicación.

Cuando se ejecute con:
python app.py
y se abra en el navegador http://127.0.0.1:5000/, deberá verse el contenido de tu index.html.



2.1.7 Crear la plantilla base con Jinja2

Crear el archivo templates/**base.html**

Archivo: templates/base.html

```
<!DOCTYPE html>
<html lang="es">
```

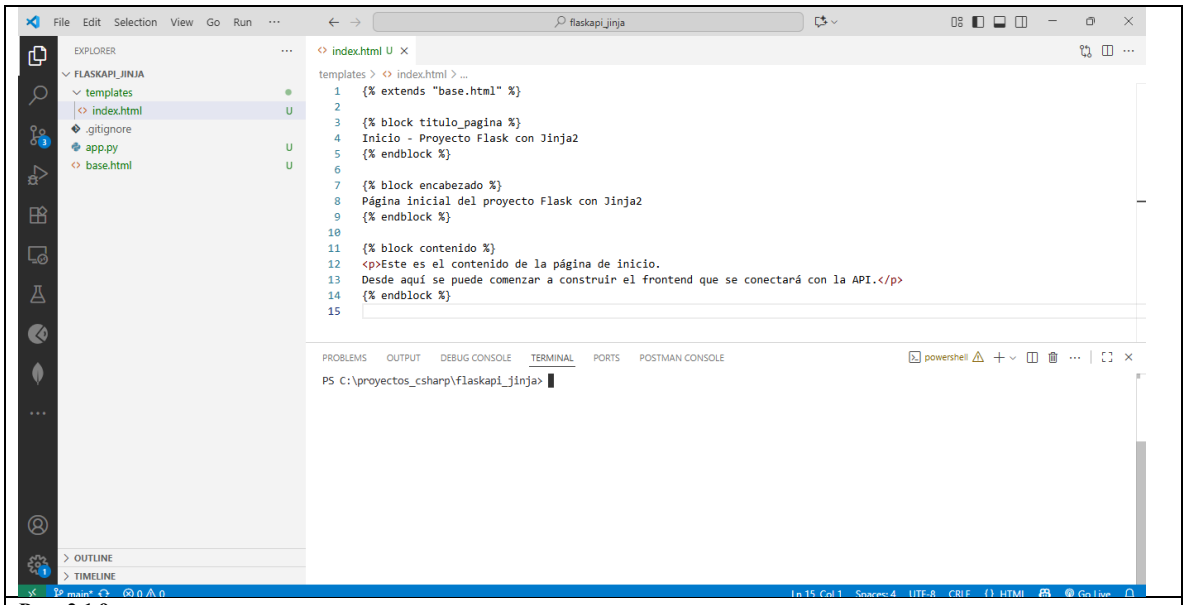
```
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <!-- Bloque de título que puede ser sobrescrito por cada página -->
   <title>{% block titulo_pagina %}Proyecto Flask con Jinja2{% endblock %}</title>

   <!-- Estilos globales opcionales -->
   <link rel="stylesheet" href="{{ url_for('static', filename='estilos.css') }}">

   <!-- Bloque opcional para que cada vista agregue estilos propios -->
   {% block estilos_extra %}{% endblock %}
</head>
<body>
   <header>
      <!-- Encabezado general de la aplicación -->
      <h1>{% block encabezado %}Aplicación Flask con Jinja2{% endblock %}</h1>

      <!-- Navegación mínima -->
      <nav>
         <a href="{{ url_for('inicio') }}">Inicio</a>
      </nav>
   </header>

   <main>
      <!-- Mensajes opcionales que se pueden pasar desde las rutas -->
      {% if mensaje %}
         <div class="mensaje">{{ mensaje }}</div>
      {% endif %}

      <!-- Contenido principal que cada página debe definir -->
      {% block contenido %}{% endblock %}
   </main>

   <footer>
      <small>&copy; {{ 2025 }} Proyecto de ejemplo</small>
   </footer>

   <!-- Bloque opcional para insertar scripts por página -->
   {% block scripts_extra %}{% endblock %}
</body>
</html>
```

Este archivo **no se ejecuta solo**, pero será la base para que index.html (y las demás vistas) extiendan su estructura y definan solo las partes necesarias.

paso 2.1.8: reescribir index.html para que herede de base.html.

```
{% extends "base.html" %}

{% block titulo_pagina %}
Inicio - Proyecto Flask con Jinja2
{% endblock %}

{% block encabezado %}
Página inicial del proyecto Flask con Jinja2
{% endblock %}

{% block contenido %}
<p>Este es el contenido de la página de inicio.
Desde aquí se puede comenzar a construir el frontend que se conectará con la API.</p>
{% endblock %}
```

{% extends "base.html" %} indica que esta plantilla reutiliza la estructura definida en base.html.
Con {% block titulo_pagina %}, {% block encabezado %} y {% block contenido %} se sobreescriben las secciones de la plantilla base.
Si en el futuro se agregan otras vistas, solo se cambia el contenido de los bloques, manteniendo un diseño uniforme.



Paso 2.1.9

1. Crear la carpeta static en la raíz del proyecto (al mismo nivel que templates).
2. Dentro de static, crear un archivo estilos.css con contenido inicial.

Archivo: static/estilos.css

```
/* Estilos generales del proyecto */

/* Se define tipografía base */
body {
   font-family: Arial, Helvetica, sans-serif;
   margin: 20px;
   background-color: #f4f4f9;
   color: #333;
}

/* Encabezado principal */
h1 {
   color: #2c3e50;
   border-bottom: 2px solid #2c3e50;
   padding-bottom: 5px;
}

/* Párrafos */
p {
   font-size: 16px;
   line-height: 1.5;
}
```

Ajuste en base.html

Hay que enlazar la hoja de estilos usando url_for('static', filename='estilos.css').

```
<!DOCTYPE html>
<html lang="es">
<head>
   <meta charset="UTF-8">
   <title>{% block titulo_pagina %}{% endblock %}</title>
   <!-- Vincular hoja de estilos ubicada en la carpeta "static" -->
   <link rel="stylesheet" href="{{ url_for('static', filename='estilos.css') }}">
</head>
<body>
   <!-- Encabezado principal -->
   <header>
      <h1>{% block encabezado %}{% endblock %}</h1>
   </header>

   <!-- Contenido específico de cada página -->
```

```
    <main>
        {% block contenido %} {% endblock %}
    </main>
</body>
</html>
```

Este archivo actúa como plantilla base.

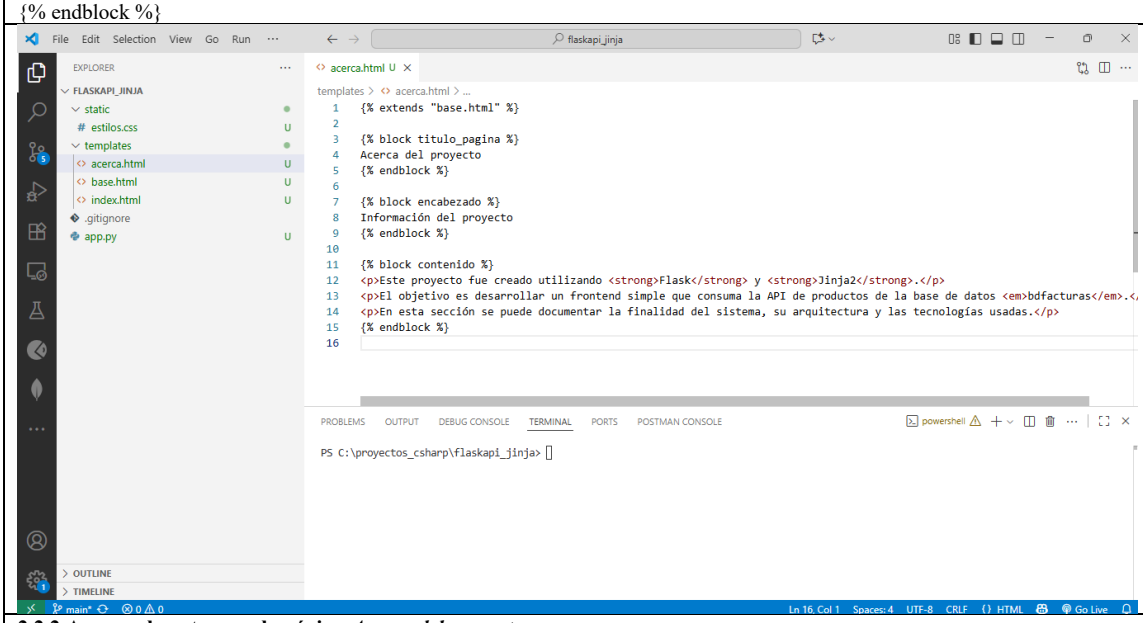
- Todo lo que escriba en base.html se hereda automáticamente en las demás páginas.
- Cada página concreta (por ejemplo index.html) solo debe rellenar los **bloques** titulo_pagina, encabezado y contenido.

2.2 Crear la página *Acerca del proyecto*

2.2.1 Crear el archivo acerca.html en la carpeta templates

Código completo del archivo:

```
{% extends "base.html" %}

{% block titulo_pagina %}
Acerca del proyecto
{% endblock %}

{% block encabezado %}
Información del proyecto
{% endblock %}

{% block contenido %}
<p>Este proyecto fue creado utilizando <strong>Flask</strong> y <strong>Jinja2</strong>.</p>
<p>El objetivo es desarrollar un frontend simple que consuma la API de productos de la base de datos <em>bdfacturas</em>.</p>
<p>En esta sección se puede documentar la finalidad del sistema, su arquitectura y las tecnologías usadas.</p>
{% endblock %}
```



2.2.2 Agregar la ruta para la página *Acerca del proyecto* en app.py

Se edita el archivo app.py y se añade una nueva ruta.

El código quedaría así (incluyendo lo que ya estaba de la página inicial):

```
# app.py
# ---------------------------------------------------------
# Archivo principal del proyecto Flask
# Aquí se configuran las rutas y la lógica inicial
# ---------------------------------------------------------

from flask import Flask, render_template

# Se crea la aplicación Flask
aplicacion = Flask(__name__)

# ---------------------------------------------------------
# Ruta para la página de inicio
```

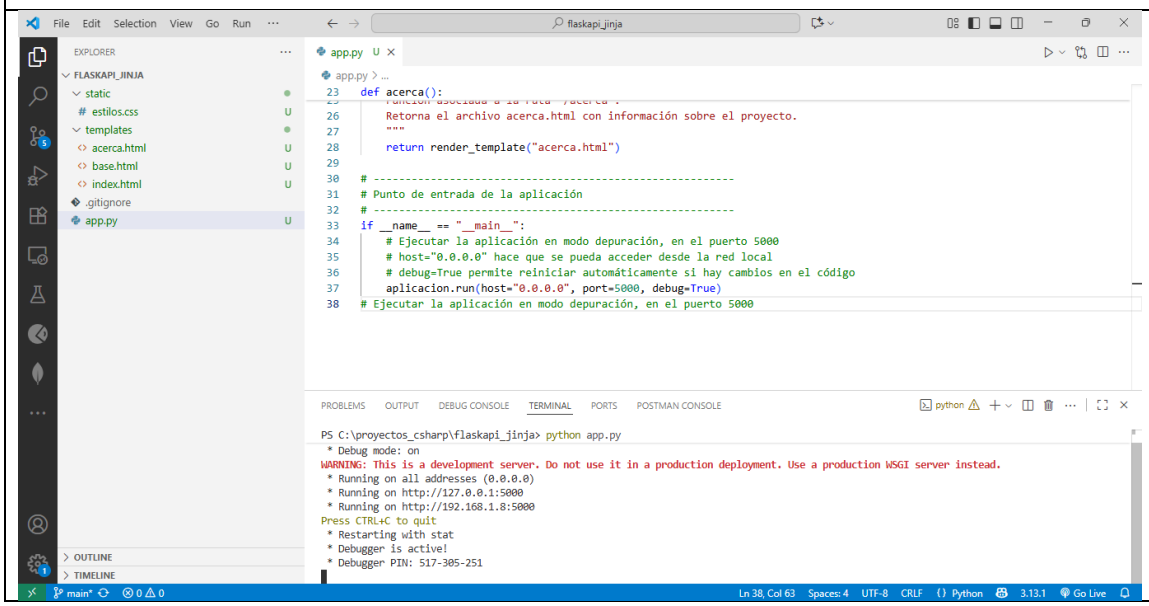
```
# ----------------------------------------------------------
@aplicacion.route("/")
def inicio():
    # Renderiza la plantilla index.html
    return render_template("index.html")

# ----------------------------------------------------------
# Ruta para la página "Acerca del proyecto"
# ----------------------------------------------------------
@aplicacion.route("/acerca")
def acerca():
    # Renderiza la plantilla acerca.html
    return render_template("acerca.html")

# ----------------------------------------------------------
# Punto de entrada de la aplicación
# ----------------------------------------------------------
if __name__ == "__main__":
    # Se ejecuta la aplicación en el puerto 5000
    aplicacion.run(debug=True, port=5000)
```

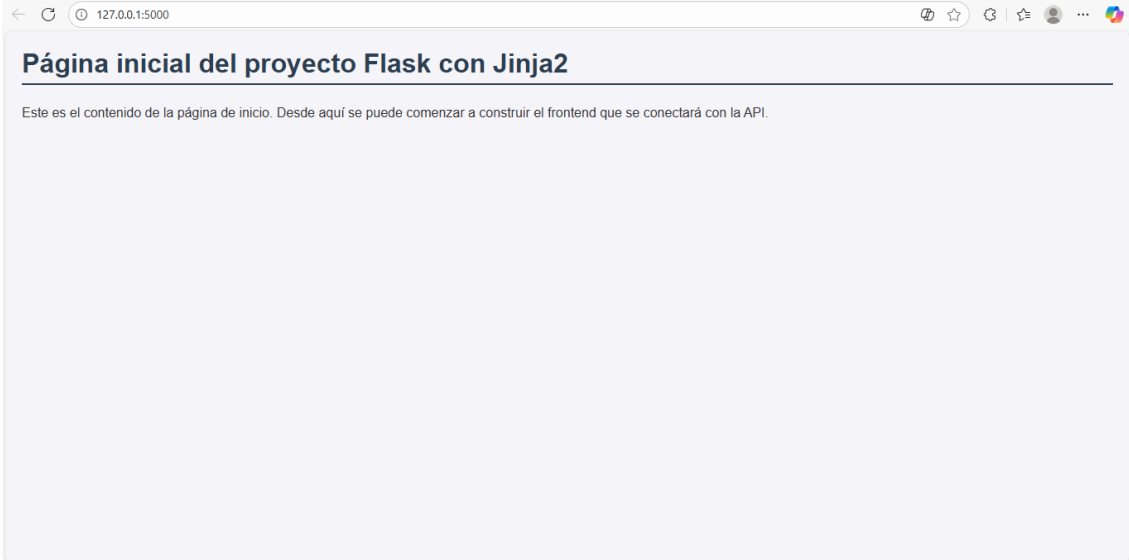
Ahora al correr:

python app.py

tendrá disponible:

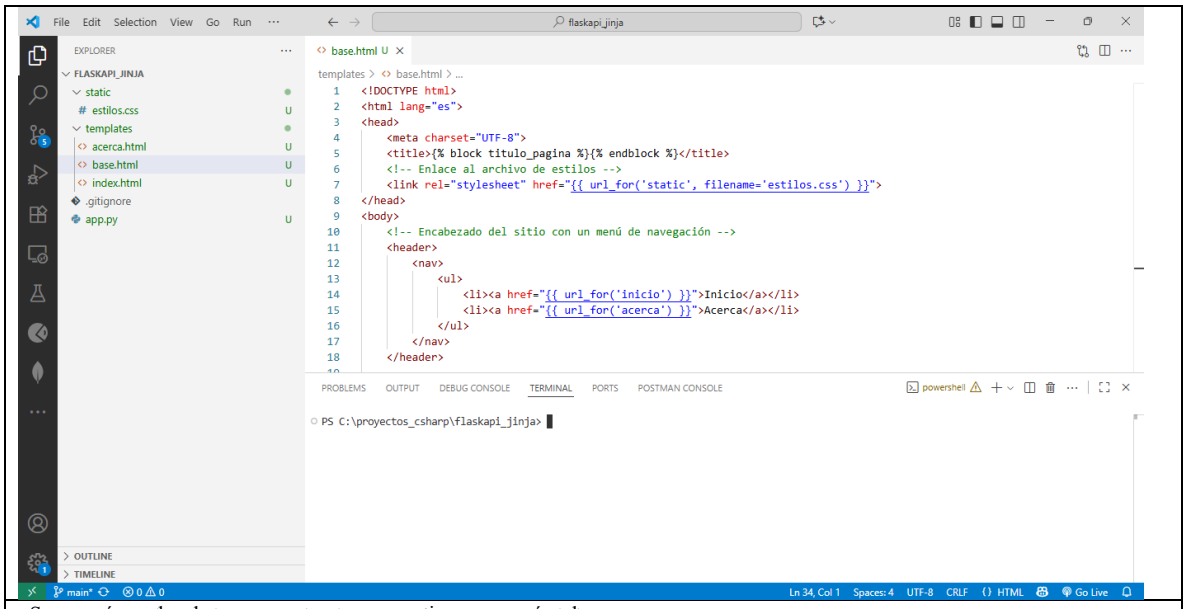
- http://127.0.0.1:5000/ → Página inicial
- http://127.0.0.1:5000/acerca → Página acerca del proyecto





Paso 2.3: Crear un menú de navegación en base.html
Se va a modificar el archivo base.html para incluir un **encabezado común** en todas las páginas.
El menú estará arriba, con enlaces a las rutas / (inicio) y /acerca.

```
<!DOCTYPE html>
<html lang="es">
<head>
   <meta charset="UTF-8">
   <title>{% block titulo_pagina %}{% endblock %}</title>
   <!-- Enlace al archivo de estilos -->
   <link rel="stylesheet" href="{{ url_for('static', filename='estilos.css') }}">
</head>
<body>
   <!-- Encabezado del sitio con un menú de navegación -->
   <header>
      <nav>
         <ul>
            <li><a href="{{ url_for('inicio') }}">Inicio</a></li>
            <li><a href="{{ url_for('acerca') }}">Acerca</a></li>
         </ul>
      </nav>
   </header>

   <!-- Sección principal de la página -->
   <main>
      <h1>{% block encabezado %}{% endblock %}</h1>
      <section>
         {% block contenido %}{% endblock %}
      </section>
   </main>

   <!-- Pie de página -->
   <footer>
      <p>Proyecto Flask con Jinja2 - © 2025</p>
   </footer>
</body>
</html>
```



Se agregó un <header> con un <nav> que contiene un menú <ul>.
Dentro del menú hay dos enlaces:

- {{ url_for('inicio') }} → apunta a la función inicio() en app.py.
- {{ url_for('acerca') }} → apunta a la función acerca() en app.py.

Con esto, el menú aparecerá en todas las páginas que hereden de base.html.

Paso 2.4: Crear static/estilos.css

Este archivo ya existe en tu proyecto, pero ahora se va a completar con reglas para:

- El cuerpo de la página.
- El menú de navegación.
- El pie de página.
- Un estilo base para títulos y párrafos.

```
/* ------------------------------
  Estilos generales del sitio
--------------------------------*/
html, body {
   height: 100%;            /* Forzar que ocupen toda la altura */
   margin: 0;
   display: flex;
   flex-direction: column;    /* Header - Main - Footer en columna */
   font-family: Arial, sans-serif;
   background-color: #f5f5f5;
   color: #333;
}

/* ------------------------------
  Estilos del encabezado y menú
--------------------------------*/
header {
   background-color: #2c3e50;
   padding: 10px 0;
}

nav ul {
   list-style-type: none;
   margin: 0;
   padding: 0;
   display: flex;
   justify-content: center;
}

nav ul li {
   margin: 0 15px;
```

```
}

nav ul li a {
    text-decoration: none;
    color: white;
    font-weight: bold;
    transition: color 0.3s ease;
}

nav ul li a:hover {
    color: #1abc9c; /* Verde azulado al pasar el ratón */
}

/* ------------------------------
   Estilos de la sección principal
-------------------------------*/
main {
    flex: 1;                /* Ocupa todo el espacio disponible */
    padding: 20px;
    text-align: center;
}

h1 {
    color: #2c3e50;
    margin-bottom: 10px;
    border-bottom: 2px solid #2c3e50;
    display: inline-block;
    padding-bottom: 5px;
}

/* ------------------------------
   Estilos del pie de página
-------------------------------*/
footer {
    background-color: #2c3e50;
    color: white;
    text-align: center;
    padding: 10px 0;
    margin-top: auto;         /* Empuja el footer al fondo */
}
```

El body usa un fondo gris claro y tipografía estándar.
El header tiene un fondo azul oscuro (#2c3e50) y dentro el menú está centrado con flexbox.
Los enlaces (a) son blancos y al pasar el cursor cambian a verde azulado (#1abc9c).
El h1 tiene una línea decorativa inferior.
El footer usa el mismo fondo oscuro que el encabezado para mantener coherencia
Se agregó html, body { height: 100%; display: flex; flex-direction: column; }.
En main se añadió flex: 1; para ocupar el espacio restante.
En footer se cambió margin-top: 20px; por margin-top: auto; para que siempre se pegue abajo

verificación del **paso 2.4**.

1. Guarda el archivo static/**estilos.css** con el contenido que te pasé.
2. Abre el archivo base.html y asegúrate de que en la sección <head> esté la referencia al CSS:

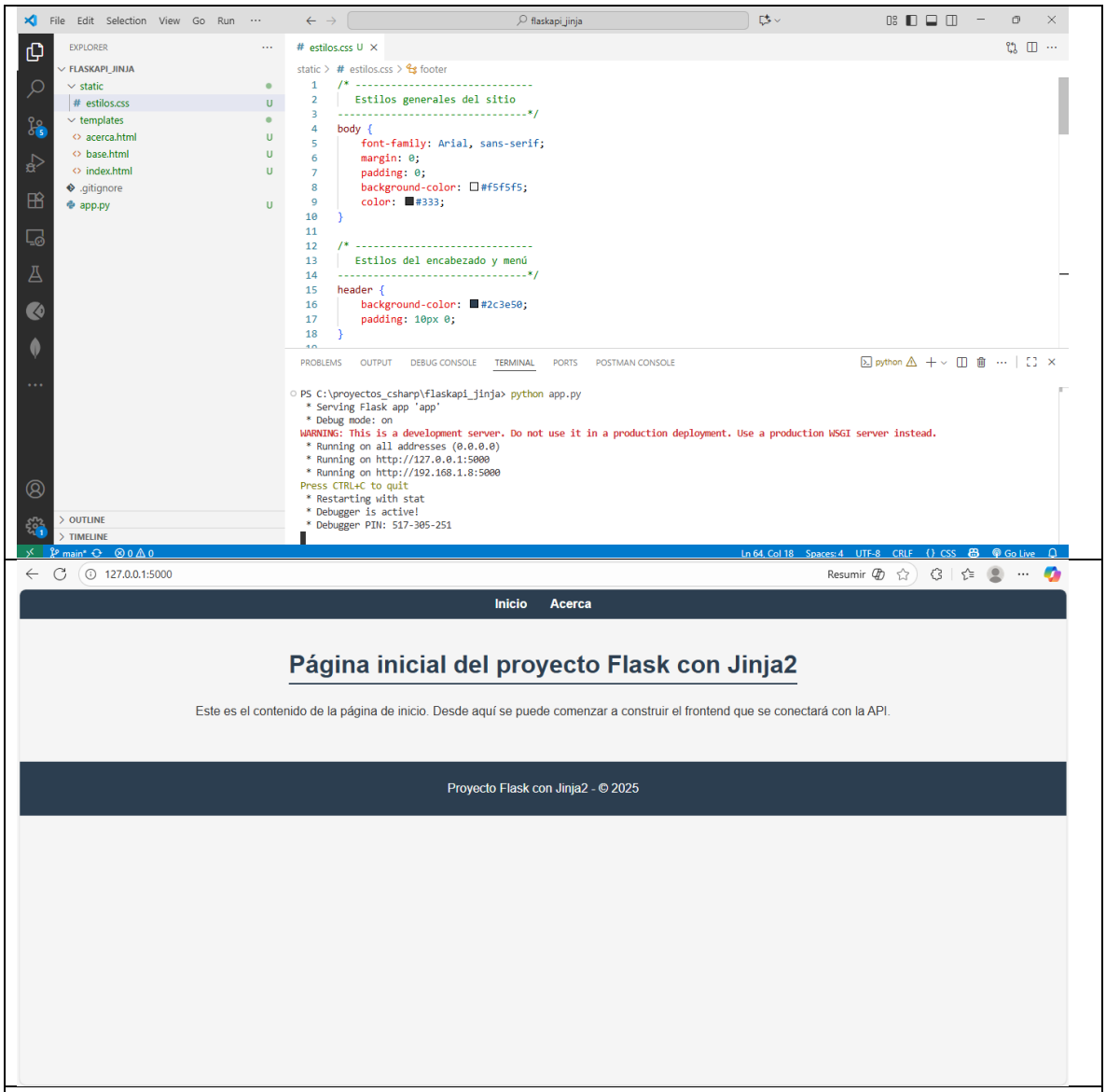
```
<link rel="stylesheet" href="{{ url_for('static', filename='estilos.css') }}">
```

3. Inicia la aplicación otra vez con:

```
python app.py
```

4. Entra al navegador en la ruta:

http://127.0.0.1:5000/



paso 2.5: mejorar la navegación.

Aquí vamos a:

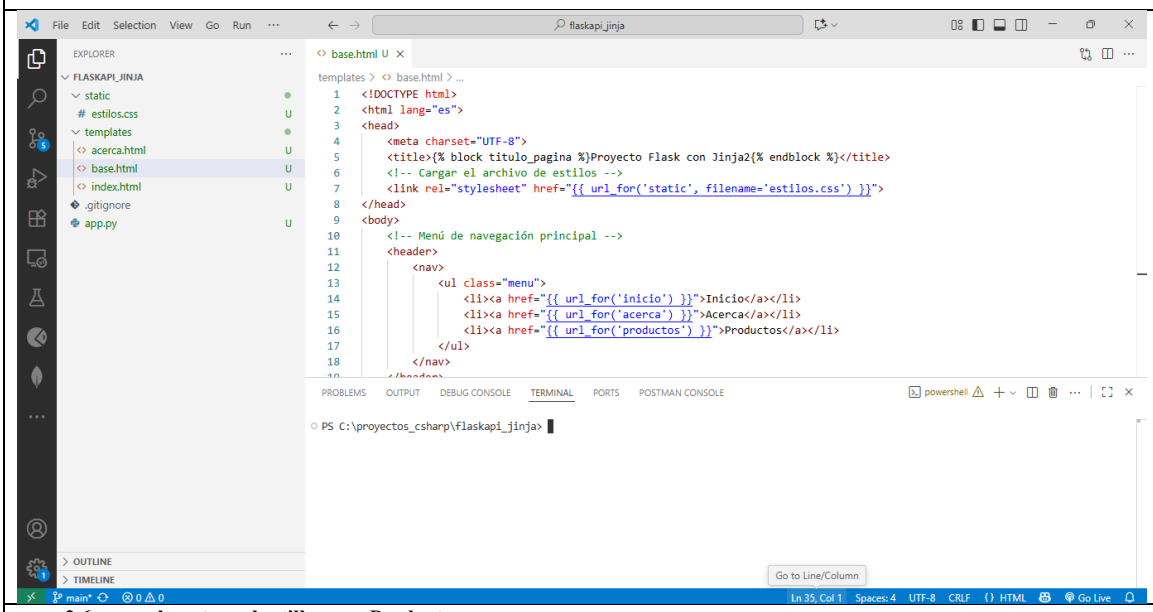
1. Dejar fijo el menú arriba y el footer abajo, dentro de base.html.
2. Agregar un nuevo enlace **Productos** en el menú para la futura página de productos.
3. Mantener consistencia en los estilos con el archivo estilos.css.

Reescribir **base.html**

```
<!DOCTYPE html>
<html lang="es">
<head>
   <meta charset="UTF-8">
   <title>{% block titulo_pagina %}Proyecto Flask con Jinja2{% endblock %}</title>
   <!-- Cargar el archivo de estilos -->
   <link rel="stylesheet" href="{{ url_for('static', filename='estilos.css') }}">
</head>
<body>
   <!-- Menú de navegación principal -->
   <header>
      <nav>
         <ul class="menu">
            <li><a href="{{ url_for('inicio') }}">Inicio</a></li>
            <li><a href="{{ url_for('acerca') }}">Acerca</a></li>
            <li><a href="{{ url_for('rutas_productos.productos') }}">Productos</a></li>
```

```
            </ul>
        </nav>
    </header>

    <!-- Contenido principal -->
    <main>
        <h1>{% block encabezado %} {% endblock %}</h1>
        <section>
            {% block contenido %} {% endblock %}
        </section>
    </main>

    <!-- Pie de página -->
    <footer>
        <p>Proyecto Flask con Jinja2 - © 2025</p>
    </footer>
</body>
</html>
```

Se añadió el enlace **Productos**:
<li><a href="{{ url_for('productos') }}">Productos</a></li>
El menú (<header>) y el footer (<footer>) están en sus lugares correctos y centralizados.
Mantiene los bloques {% block ... %} para que cada vista herede de este diseño base.



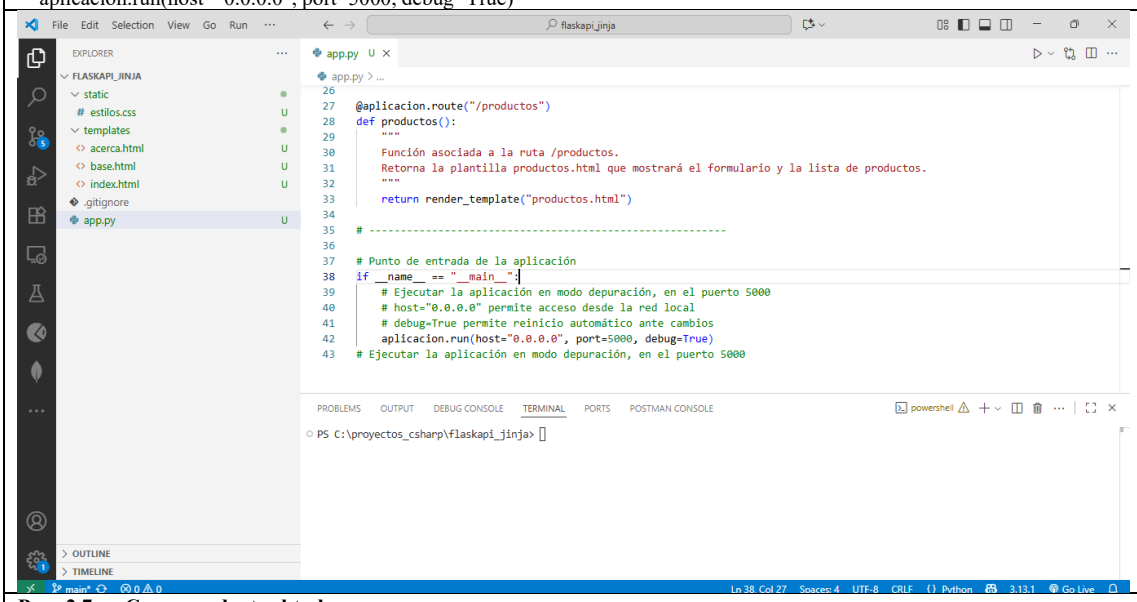
paso 2.6: crear la ruta y plantilla para Productos.
En este paso se va a:

1. Agregar la ruta /productos en app.py.
2. Crear la plantilla productos.html heredando de base.html.
3. Dejar un espacio preparado donde luego se pondrá el formulario que consuma la API.

Paso 2.6 — Modificar app.py

```
# Importar la clase principal de Flask y la función para renderizar plantillas
from flask import Flask, render_template

# Crear la instancia de la aplicación Flask
aplicacion = Flask(__name__)

# ------------------- Rutas principales -------------------

@aplicacion.route("/")
def inicio():
    """
    Función asociada a la ruta principal (/).
    Retorna la plantilla index.html.
    """
```

```
    return render_template("index.html")


@aplicacion.route("/acerca")
def acerca():
    """
    Función asociada a la ruta /acerca.
    Retorna la plantilla acerca.html con información sobre el proyecto.
    """
    return render_template("acerca.html")


@aplicacion.route("/productos")
def productos():
    """
    Función asociada a la ruta /productos.
    Retorna la plantilla productos.html que mostrará el formulario y la lista de productos.
    """
    return render_template("productos.html")

# -------------------------------------------------------

# Punto de entrada de la aplicación
if __name__ == "__main__":
    # Ejecutar la aplicación en modo depuración, en el puerto 5000
    # host="0.0.0.0" permite acceso desde la red local
    # debug=True permite reinicio automático ante cambios
    aplicacion.run(host="0.0.0.0", port=5000, debug=True)
```



Paso 2.7 — Crear productos.html
Archivo en **templates/productos.html**:

```
{% extends "base.html" %}

{% block titulo_pagina %}
Productos
{% endblock %}

{% block encabezado %}
Gestión de Productos
{% endblock %}

{% block contenido %}
<p>Aquí se mostrará el formulario para crear, buscar, actualizar y eliminar productos.</p>
<p>También se listarán todos los productos obtenidos desde la API.</p>
{% endblock %}
```

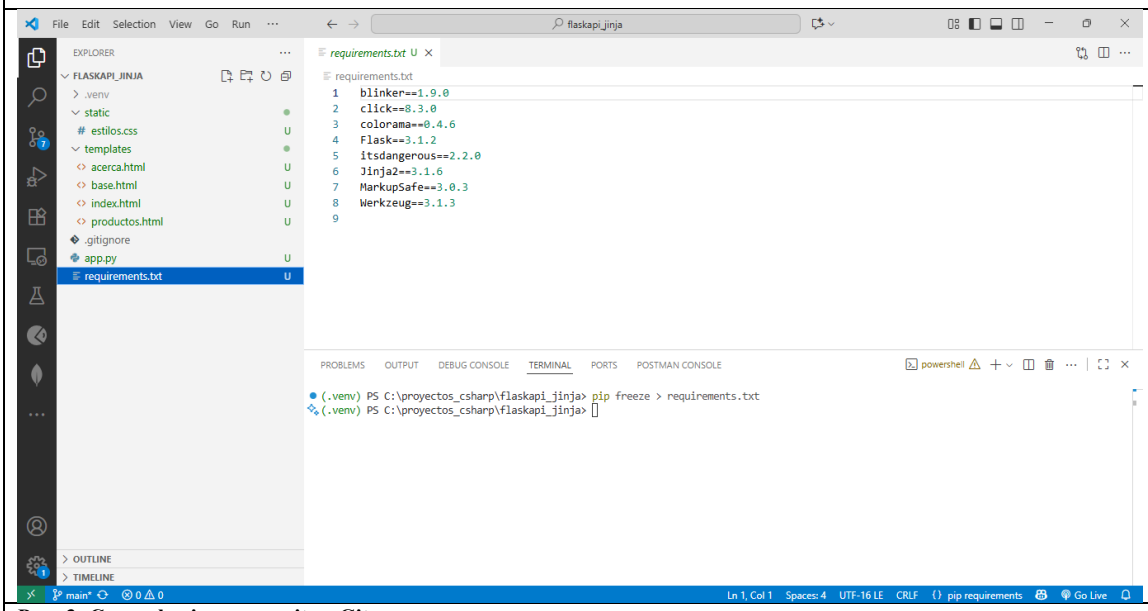
2.8 Congelar dependencias en requirements.txt

Ahora que Flask está instalado en el entorno virtual, conviene guardar todas las dependencias exactas en un archivo requirements.txt. Este archivo permite que cualquier otra persona (o usted mismo en otro equipo) pueda recrear el mismo entorno.

Comando en PowerShell (con .venv activo):

pip freeze > requirements.txt

Esto creará el archivo requirements.txt en la raíz del proyecto con las versiones exactas de Flask y librerías asociadas.



Paso 3: Crear el primer commit en Git

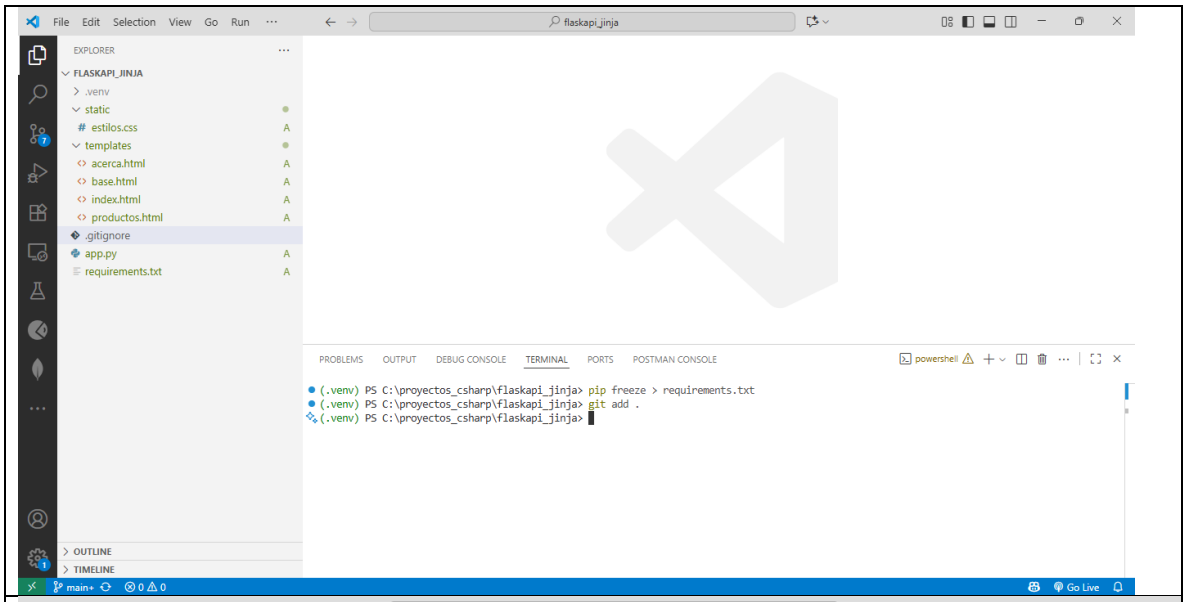
En la terminal de PowerShell (dentro de VSCode y en la raíz del proyecto), ejecutar:

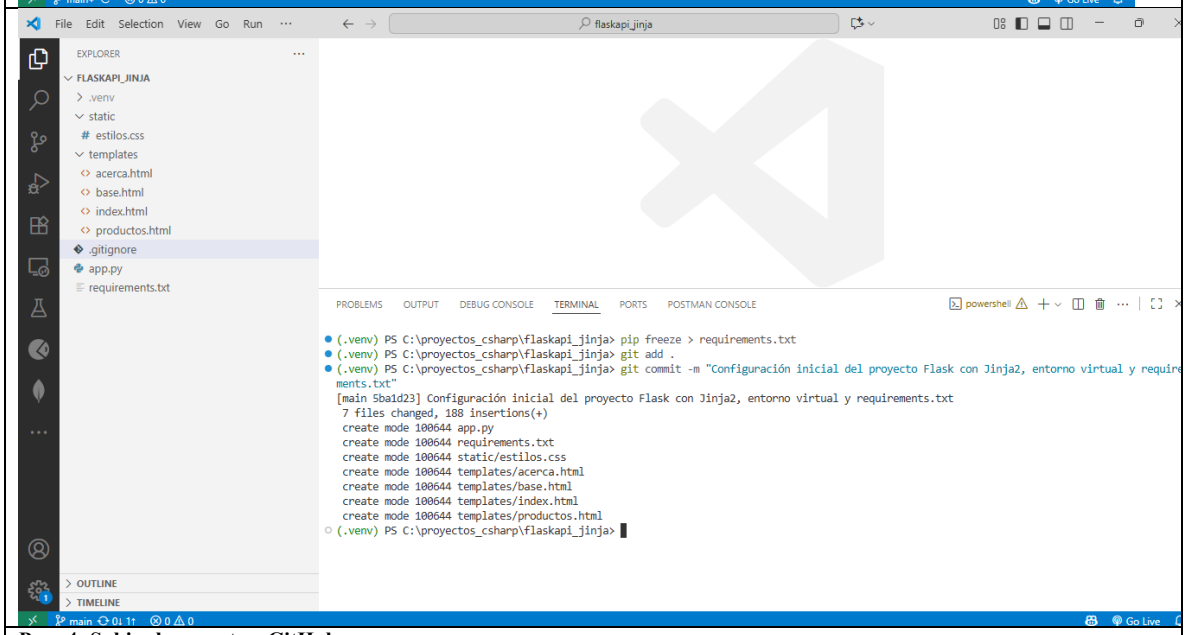
git add .

git commit -m "Configuración inicial del proyecto Flask con Jinja2, entorno virtual y requirements.txt"

1. **git add .**
   - Indica a Git que agregue todos los archivos nuevos y modificados al *staging area* (área de preparación).
   - El . significa "todo el proyecto actual".
2. **git commit -m "mensaje"**
   - Crea un punto de control en el historial del repositorio con esos cambios.
   - El mensaje debe ser claro y descriptivo.
   - Ejemplo:
     "Configuración inicial del proyecto Flask con Jinja2, entorno virtual y requirements.txt"

Después de este commit ya queda un **primer registro histórico** de su proyecto.





Paso 4: Subir el proyecto a GitHub

1. En la terminal, asegurarse de estar en la carpeta del proyecto (ya está en frontflask_jinja).
2. Ejecutar este comando para conectar el repositorio local con el remoto en GitHub:

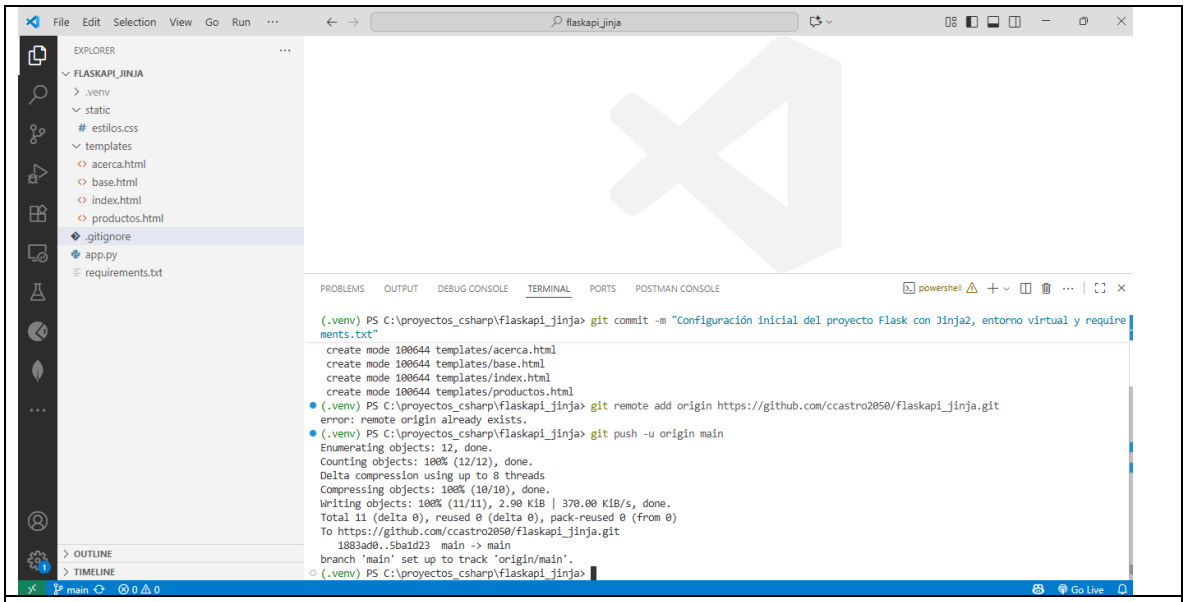
git remote add origin https://github.com/ccastro2050/frontflask_jinja.git

3. Luego subir la rama principal main al repositorio remoto:

git push -u origin main

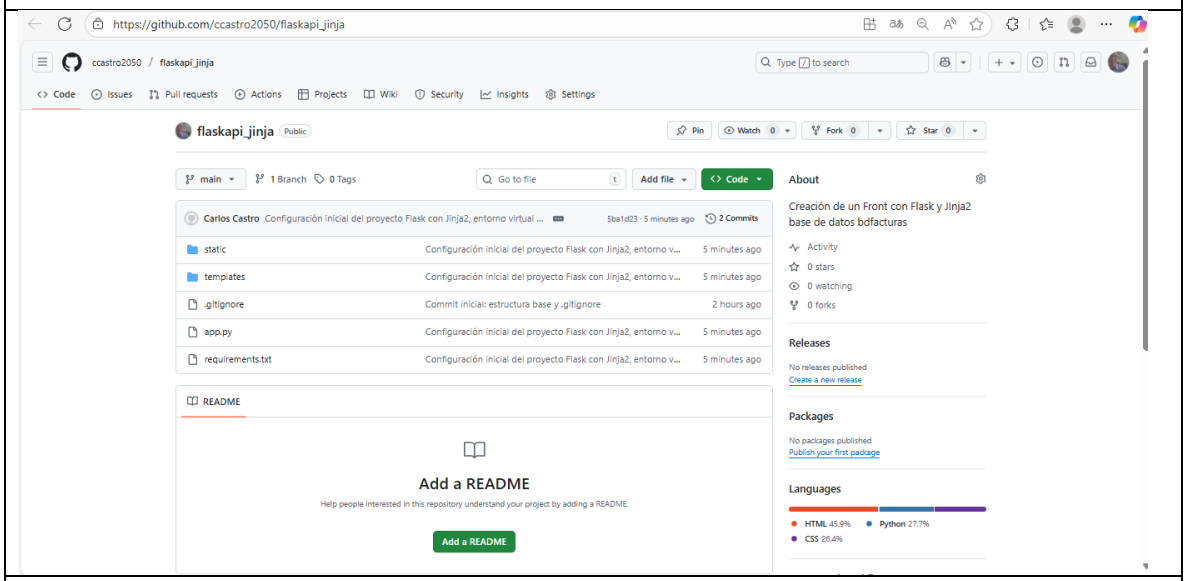
Explicación

- **git remote add origin URL**
  - Asocia el repositorio local con el repositorio remoto en GitHub (enlace entre ambos).
  - Solo se hace una vez.
- **git push -u origin main**
  - Envía los commits de la rama main al repositorio remoto.
  - La opción -u guarda la configuración para que en el futuro solo sea necesario usar git push.



Cómo verificar en GitHub

1. Abrir en el navegador el enlace de tu repositorio:
   https://github.com/ccastro2050/frontflask_jinja
2. En la interfaz de GitHub deberías ver:
   - Los archivos del proyecto (app.py, requirements.txt, carpeta templates, carpeta static, .gitignore).
   - El mensaje de commit:
     "Configuración inicial del proyecto Flask con Jinja2, entorno virtual y requirements.txt".



Paso 5: Crear un *Blueprint* para productos

La idea de usar *Blueprints* es separar la lógica de cada módulo (productos, clientes, facturas, etc.) en archivos independientes. Así mantenemos el código más ordenado y fácil de mantener.

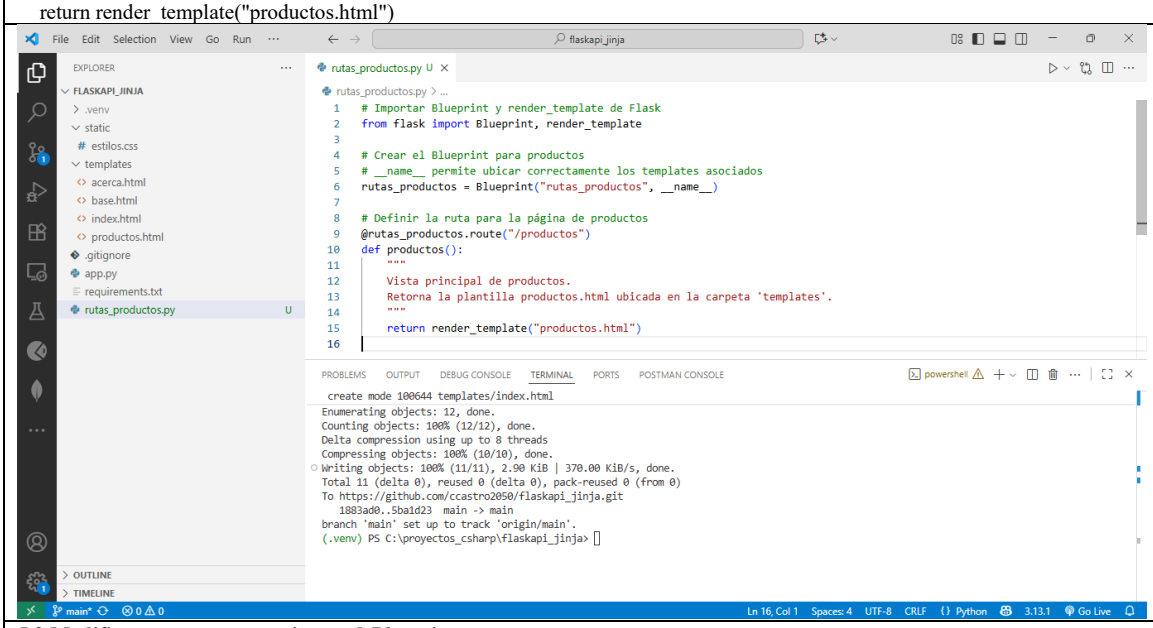
5.1 Crear archivo rutas_productos.py

Cree un nuevo archivo en la raíz del proyecto (junto a app.py):

rutas_productos.py

```
# Importar Blueprint y render_template desde Flask
from flask import Blueprint, render_template

# Crear el Blueprint para agrupar las rutas relacionadas con productos
# El primer argumento es el nombre del blueprint, y __name__ ayuda a ubicar recursos como plantillas
rutas_productos = Blueprint("rutas_productos", __name__)

# ------------------- Rutas de Productos -------------------
```

```
@rutas_productos.route("/productos")
def productos():
    """
    Función asociada a la ruta /productos.
    Retorna la plantilla productos.html, donde se mostrará el formulario
    y la lista de productos obtenidos desde la API.
    """
    return render_template("productos.html")
```



5.2 Modificar app.py para registrar el *Blueprint*

Edite su **app.py** y agregue la importación + registro del *Blueprint*:

```
# Importar la clase principal de Flask y la función para renderizar plantillas
from flask import Flask, render_template

# Importar el Blueprint que contiene las rutas de productos
from rutas_productos import rutas_productos

# Crear la instancia de la aplicación Flask
aplicacion = Flask(__name__)

# ------------------- Registro de Blueprints -------------------
# Registrar el Blueprint de productos en la aplicación principal
aplicacion.register_blueprint(rutas_productos)

# ------------------- Rutas principales -------------------

@aplicacion.route("/")
def inicio():
    """
    Función asociada a la ruta principal (/).
    Retorna la plantilla index.html.
    """
    return render_template("index.html")


@aplicacion.route("/acerca")
def acerca():
    """
    Función asociada a la ruta /acerca.
    Retorna la plantilla acerca.html con información sobre el proyecto.
    """
    return render_template("acerca.html")

# -------------------------------------------------------

# Punto de entrada de la aplicación
```

```
if __name__ == "__main__":
    # Ejecutar la aplicación en modo depuración, en el puerto 5000
    # host="0.0.0.0" permite que la app sea accesible desde la red local
    # debug=True permite reinicio automático ante cambios
    aplicacion.run(host="0.0.0.0", port=5000, debug=True)
```

Con esto, cuando abra en el navegador:

- http://127.0.0.1:5000/ → carga la página de inicio
- http://127.0.0.1:5000/productos → carga la página de productos

Con esta separación:

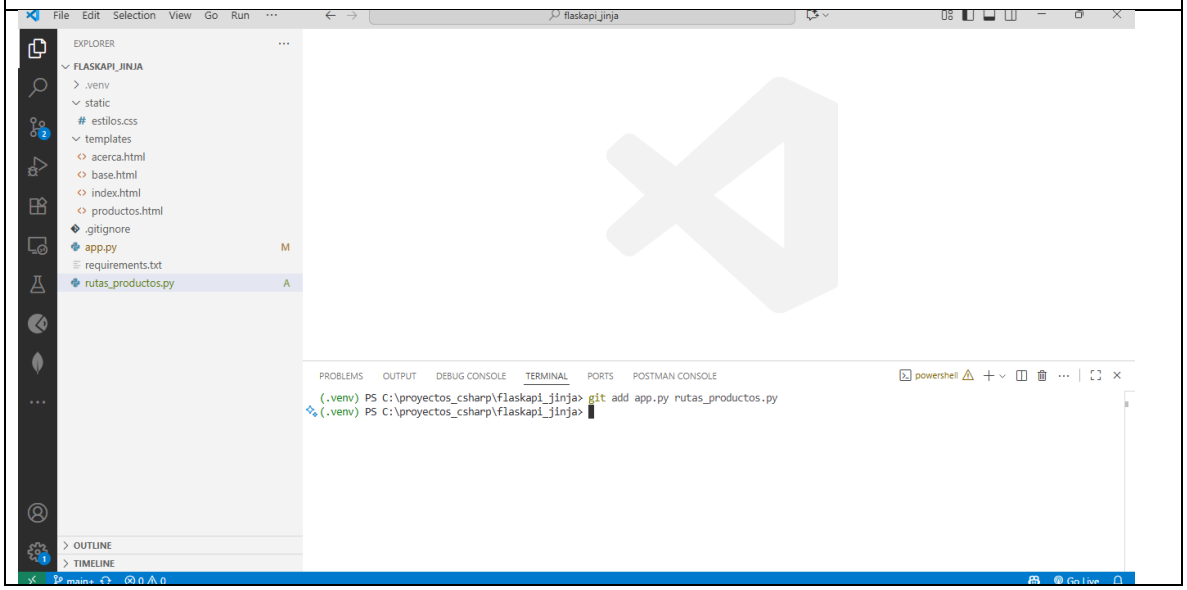
- app.py se mantiene limpio y solo organiza la aplicación.
- rutas_productos.py se encarga de todo lo relacionado con productos.
- En el futuro, puedes crear más Blueprints (ej: rutas_clientes.py, rutas_facturas.py) y registrarlos en app.py.

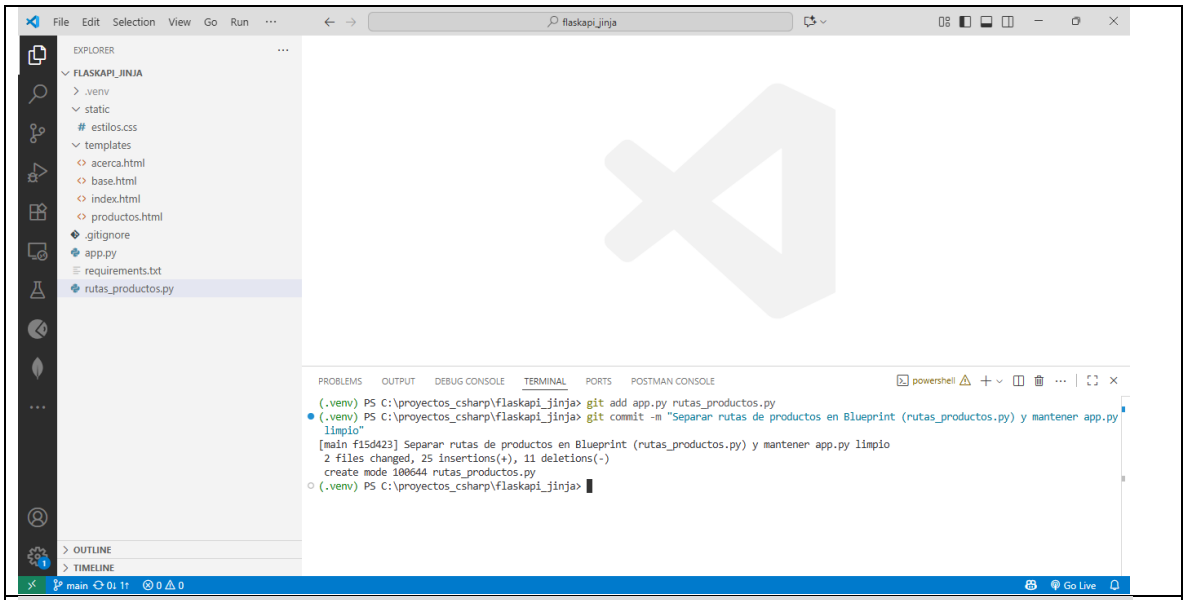
paso 5.3 será hacer el commit de esta reorganización con el **Blueprint de productos**.

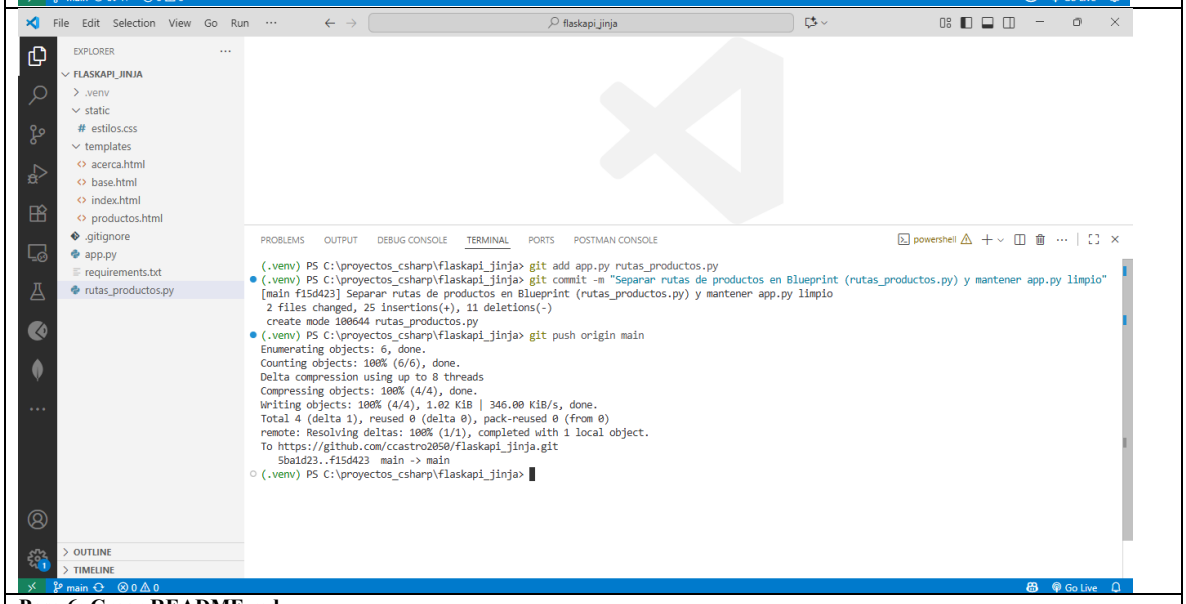
Pasos en PowerShell (ya dentro de tu entorno virtual .venv):

1. Guardar los cambios en **app.py**.
2. Crear el nuevo archivo **rutas_productos.py**
3. Agregar los cambios al control de versiones:
   git add app.py rutas_productos.py
4. Hacer el commit con un mensaje claro:
   git commit -m "Separar rutas de productos en Blueprint (rutas_productos.py) y mantener app.py limpio"
5. Subir los cambios a GitHub:
   git push origin main

Después de esto, en GitHub verás un nuevo commit con la reorganización de las rutas.





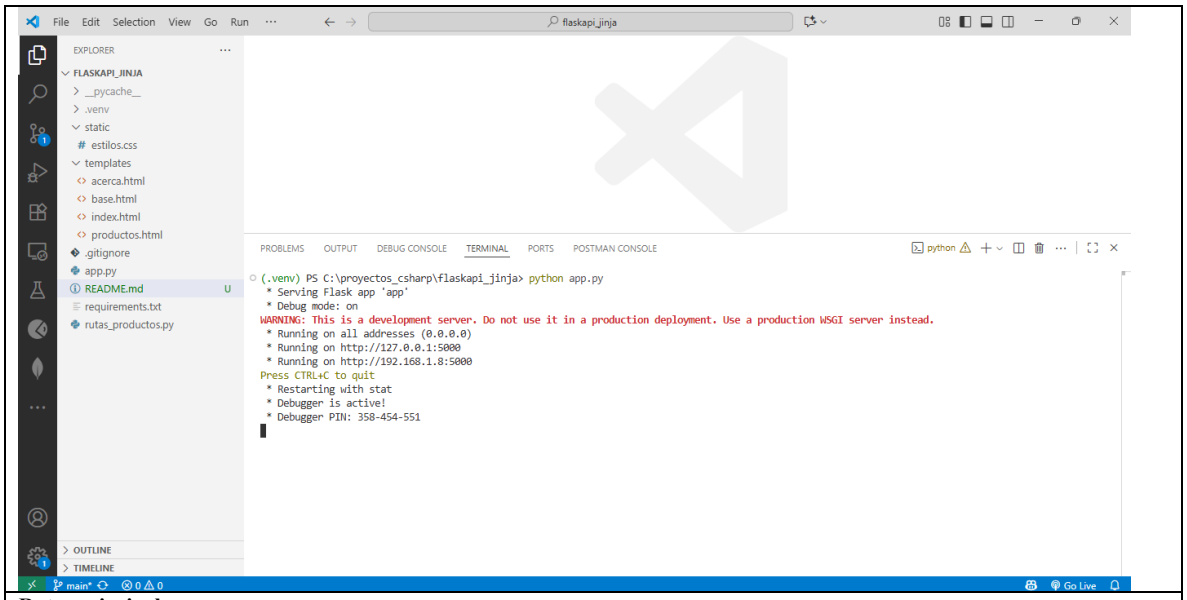


Paso 6: Crear README.md

1. En la raíz del proyecto (misma carpeta donde está app.py y requirements.txt), crear un archivo llamado: **README.md**

```
# Proyecto Flask con Jinja2

Este proyecto implementa un frontend en **Flask** utilizando el motor de plantillas **Jinja2**, con el objetivo de conectarse a una API que gestiona productos, facturas, clientes y vendedores. La aplicación se organiza con **Blueprints** para mantener un código modular y fácil de mantener.

---

## Estructura del proyecto

- **app.py**: archivo principal de la aplicación Flask. Crea la instancia de la app y registra los Blueprints.
```

```
- **rutas_productos.py**: Blueprint con las rutas de la sección de productos.
- **templates/**: plantillas HTML del proyecto.
  - `base.html` (plantilla base con encabezado, menú y pie de página)
  - `index.html` (inicio)
  - `acerca.html` (información del proyecto)
  - `productos.html` (página de productos)
- **static/**: archivos estáticos, por ejemplo `estilos.css`.
- **requirements.txt**: dependencias exactas del proyecto.

---

## Requisitos

- Python 3.11 o superior
- Git instalado
- Windows PowerShell o terminal equivalente

---

## Instalación

1. Clonar el repositorio:
   git clone https://github.com/ccastro2050/frontflask_jinja.git
   cd frontflask_jinja

2. Crear y activar el entorno virtual:
python -m venv .venv
.\.venv\Scripts\activate

3. Instalar dependencias:
pip install -r requirements.txt

4. Ejecución

Iniciar el servidor de desarrollo:
python app.py
```



Rutas principales

- / → Página de inicio
- /acerca → Información del proyecto
- /productos → Sección de productos (vista servida por el Blueprint rutas_productos)

paso 7: Construir un **CRUDL completo para productos**, bien integrado, con el formulario dinámico y la tabla de listado.
La idea es que quede **un solo formulario** que sirva tanto para **crear** como para **actualizar**, que pueda **buscar** por ID y que en la tabla se muestre la lista con botones de **eliminar**.

Paso 7.1 – rutas_productos.py
Creamos un Blueprint con todas las operaciones del CRUDL.

```
# Importar módulos necesarios de Flask y la librería requests para conectarse a la API externa
from flask import Blueprint, render_template, request, redirect, url_for
import requests

# Crear el Blueprint de productos
# "rutas_productos" es el nombre del módulo
# __name__ permite ubicar las plantillas dentro del proyecto
rutas_productos = Blueprint("rutas_productos", __name__)

# URL base de la API en C# que gestiona los productos
```

```
API_URL = "http://localhost:5031/api/producto"

# ------------------- LISTAR productos -------------------
@rutas_productos.route("/productos")
def productos():
    """
    Ruta para listar todos los productos disponibles en la API.
    Llama al endpoint /api/producto y extrae la lista de productos desde la clave "datos".
    """
    try:
        respuesta = requests.get(API_URL)
        productos = respuesta.json().get("datos", [])
    except Exception as e:
        productos = []
        print("Error al conectar con la API:", e)

    # Retorna la plantilla productos.html con la lista de productos
    return render_template(
        "productos.html",
        productos=productos,
        producto=None,
        modo="crear"
    )

# ------------------- BUSCAR producto -------------------
@rutas_productos.route("/productos/buscar", methods=["POST"])
def buscar_producto():
    """
    Ruta para buscar un producto específico a partir de su código.
    Si lo encuentra, carga sus datos en el formulario en modo "actualizar".
    Si no existe, retorna un mensaje y la lista completa.
    """
    codigo = request.form.get("codigo_buscar")

    if codigo:
        try:
            respuesta = requests.get(f"{API_URL}/codigo/{codigo}")
            if respuesta.status_code == 200:
                datos = respuesta.json().get("datos", [])
                if datos:
                    # Si la API retorna datos, se asume que es una lista con un producto
                    producto = datos[0]
                    productos = requests.get(API_URL).json().get("datos", [])
                    return render_template(
                        "productos.html",
                        productos=productos,
                        producto=producto,
                        modo="actualizar"
                    )
        except Exception as e:
            return f"Error en la búsqueda: {e}"

    # Si no se encuentra el producto, recargar la lista completa
    productos = requests.get(API_URL).json().get("datos", [])
    return render_template(
        "productos.html",
        productos=productos,
        producto=None,
        mensaje="Producto no encontrado",
        modo="crear"
    )

# ------------------- CREAR producto -------------------
@rutas_productos.route("/productos/crear", methods=["POST"])
def crear_producto():
    """
    Ruta para crear un producto nuevo en la API.
    Toma los datos del formulario y los envía vía POST al endpoint /api/producto.
    """
    datos = {
```

```
        "codigo": request.form.get("codigo"),
        "nombre": request.form.get("nombre"),
        "valorunitario": int(request.form.get("valorunitario", 0)),
        "stock": int(request.form.get("stock", 0))
    }

    try:
        requests.post(API_URL, json=datos)
    except Exception as e:
        return f"Error al crear producto: {e}"

    return redirect(url_for("rutas_productos.productos"))

# ------------------- ACTUALIZAR producto -------------------
@rutas_productos.route("/productos/actualizar", methods=["POST"])
def actualizar_producto():
    """
    Ruta para actualizar un producto existente en la API.
    Busca por código y realiza una petición PUT con los nuevos datos.
    """
    codigo = request.form.get("codigo")
    datos = {
        "nombre": request.form.get("nombre"),
        "valorunitario": int(request.form.get("valorunitario", 0)),
        "stock": int(request.form.get("stock", 0))
    }

    try:
        requests.put(f"{API_URL}/codigo/{codigo}", json=datos)
    except Exception as e:
        return f"Error al actualizar producto: {e}"

    return redirect(url_for("rutas_productos.productos"))

# ------------------- ELIMINAR producto -------------------
@rutas_productos.route("/productos/eliminar/<string:codigo>", methods=["POST"])
def eliminar_producto(codigo):
    """
    Ruta para eliminar un producto de la API según su código.
    Envía una petición DELETE al endpoint correspondiente.
    """
    try:
        requests.delete(f"{API_URL}/codigo/{codigo}")
    except Exception as e:
        return f"Error al eliminar producto: {e}"

    return redirect(url_for("rutas_productos.productos"))
```

Instalar requests
El frontend Flask va a consumir tu API con la librería requests.
En la terminal, dentro del entorno virtual:
pip install requests
Y luego actualiza tu requirements.txt:
pip freeze > requirements.txt

Paso 7.2 – productos.html
Un solo formulario que sirve para **crear** o **actualizar**.
Si se ha buscado un producto, se rellenan los campos automáticamente y el botón cambia.

```
{% extends "base.html" %}

{% block contenido %}
    <h1>Gestión de Productos</h1>

    <!-- ==============================
         Formulario de creación/actualización
         - Usa el modo "crear" o "actualizar" según lo que se envíe desde Flask
         - Si "producto" tiene datos, los carga en el formulario
    ================================== -->
```

```
<form method="post"
     action="{{ url_for('rutas_productos.crear_producto') if modo == 'crear' else url_for('rutas_productos.actualizar_producto') }}">

   <!-- Campo código -->
   <input type="text" name="codigo" placeholder="Código"
         value="{{ producto.codigo if producto else '' }}" required>

   <!-- Campo nombre -->
   <input type="text" name="nombre" placeholder="Nombre"
         value="{{ producto.nombre if producto else '' }}" required>

   <!-- Campo valor unitario -->
   <input type="number" name="valorunitario" placeholder="Valor unitario"
         value="{{ producto.valorunitario if producto else '' }}" required>

   <!-- Campo stock -->
   <input type="number" name="stock" placeholder="Stock"
         value="{{ producto.stock if producto else '' }}" required>

   <!-- Botón que cambia dinámicamente -->
   <button type="submit">
      {{ "Crear" if modo == "crear" else "Actualizar" }}
   </button>
</form>

<!-- ================================
    Formulario de búsqueda
    - Permite buscar un producto por su código
    - Si lo encuentra, carga sus datos en el formulario principal
==================================== -->
<form method="post" action="{{ url_for('rutas_productos.buscar_producto') }}">
   <input type="text" name="codigo_buscar" placeholder="Buscar por código" required>
   <button type="submit">Buscar</button>
</form>

<!-- Mensaje de error o notificación -->
{% if mensaje %}
   <p style="color: red;">{{ mensaje }}</p>
{% endif %}

<!-- ================================
    Tabla con todos los productos
    - Los productos vienen desde la API en la clave "datos"
    - Cada fila muestra un producto con su botón de eliminar
==================================== -->
<table border="1" cellpadding="5" cellspacing="0" style="margin: 20px auto;">
   <thead>
      <tr>
         <th>Código</th>
         <th>Nombre</th>
         <th>Valor Unitario</th>
         <th>Stock</th>
         <th>Acciones</th>
      </tr>
   </thead>
   <tbody>
      {% for p in productos %}
         <tr>
            <td>{{ p.codigo }}</td>
            <td>{{ p.nombre }}</td>
            <td>{{ p.valorunitario }}</td>
            <td>{{ p.stock }}</td>
            <td>
               <!-- Formulario para eliminar un producto -->
               <form method="post"
                     action="{{ url_for('rutas_productos.eliminar_producto', codigo=p.codigo) }}">
                  <button type="submit">Eliminar</button>
               </form>
            </td>
         </tr>
```

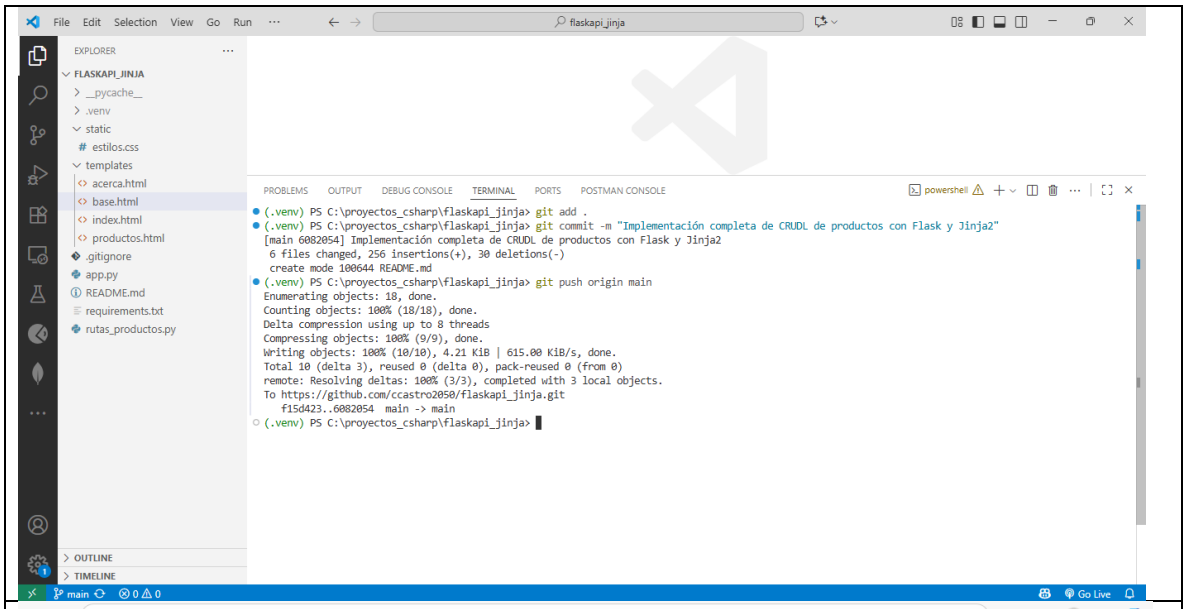
```
        {% endfor %}
      </tbody>
    </table>
{% endblock %}
```

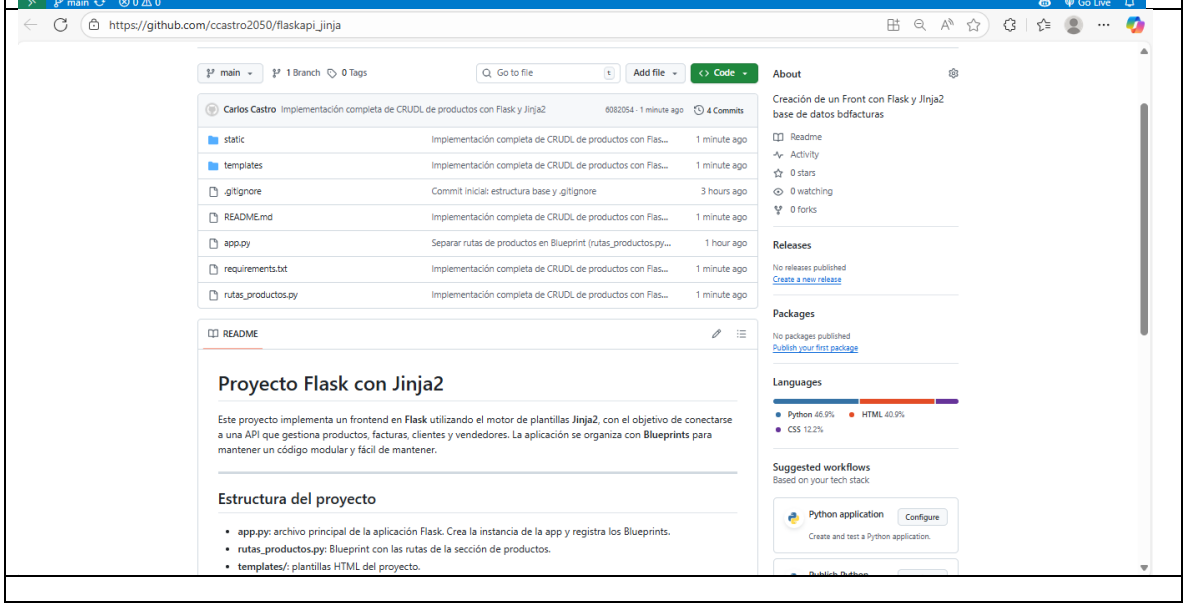
Paso 8: Registrar cambios finales en GitHub

1. Abrir la terminal en la raíz del proyecto (C:\proyectos_csharp\frontflask_jinja).
2. Agregar todos los cambios (archivos modificados y nuevos):
   git add .
3. Crear un commit descriptivo, por ejemplo:
   git commit -m "Implementación completa de CRUDL de productos con Flask y Jinja2"
   - **CRUDL** = Crear, Leer, Actualizar, Eliminar y Listar.
   - Este mensaje deja claro que el módulo de productos ya está implementado.
4. Subir los cambios al repositorio remoto en GitHub:
   git push origin main

- **git add .**: toma todos los cambios en los archivos para incluirlos en el próximo commit.
- **git commit -m "mensaje"**: guarda un "punto en la historia" con los cambios y una descripción.
- **git push origin main**: sube el commit al repositorio remoto en GitHub, en la rama main.

Una vez hecho, si entras a tu repositorio en GitHub, verás el commit más reciente con el mensaje que hayas escrito.


File Edit Selection View Go Run
flaskapi_jinja
EXPLORER
FLASKAPI_JINJA
__pycache__
.venv
static
estilos.css
templates
acerca.html
base.html
index.html
productos.html
.gitignore
app.py
README.md
requirements.txt
rutas_productos.py
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
powershell
(.venv) PS C:\proyectos_csharp\flaskapi_jinja> git add .
(.venv) PS C:\proyectos_csharp\flaskapi_jinja> git commit -m "Implementación completa de CRUDL de productos con Flask y Jinja2"
[main 6082054] Implementación completa de CRUDL de productos con Flask y Jinja2
6 files changed, 256 insertions(+), 30 deletions(-)
create mode 100644 README.md
(.venv) PS C:\proyectos_csharp\flaskapi_jinja> git push origin main
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 4.21 KiB | 615.00 KiB/s, done.
Total 10 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/ccastro2050/flaskapi_jinja.git
f15d423..6082054 main -> main
(.venv) PS C:\proyectos_csharp\flaskapi_jinja>
main
Go Live


https://github.com/ccastro2050/flaskapi_jinja
main
1 Branch
0 Tags
Go to file
Add file
Code
Carlos Castro Implementación completa de CRUDL de productos con Flask y Jinja2
6082054 · 1 minute ago
4 Commits
static Implementación completa de CRUDL de productos con Flas... 1 minute ago
templates Implementación completa de CRUDL de productos con Flas... 1 minute ago
.gitignore Commit inicial: estructura base y .gitignore 3 hours ago
README.md Implementación completa de CRUDL de productos con Flas... 1 minute ago
app.py Separar rutas de productos en Blueprint (rutas_productos.py... 1 hour ago
requirements.txt Implementación completa de CRUDL de productos con Flas... 1 minute ago
rutas_productos.py Implementación completa de CRUDL de productos con Flas... 1 minute ago
README
Proyecto Flask con Jinja2
Este proyecto implementa un frontend en Flask utilizando el motor de plantillas Jinja2, con el objetivo de conectarse a una API que gestiona productos, facturas, clientes y vendedores. La aplicación se organiza con Blueprints para mantener un código modular y fácil de mantener.
Estructura del proyecto
app.py: archivo principal de la aplicación Flask. Crea la instancia de la app y registra los Blueprints.
rutas_productos.py: Blueprint con las rutas de la sección de productos.
templates/: plantillas HTML del proyecto.
About
Creación de un Front con Flask y JInja2 base de datos bdfacturas
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Languages
Python 46.9%
HTML 40.9%
CSS 12.2%
Suggested workflows
Based on your tech stack
Python application
Configure
Create and test a Python application.